
SEEDPRINTS: FINGERPRINTS CAN EVEN TELL WHICH SEED YOUR LARGE LANGUAGE MODEL WAS TRAINED FROM

Anonymous authors
Anonymous submission

ABSTRACT

Fingerprinting Large Language Models (LLMs) is a core technical requirement for provenance verification and model attribution. Existing fingerprinting methods are typically evaluated after fine-tuning, when the models under test have already accumulated stable signatures induced by training data, optimization trajectories, or hyperparameter choices. Because most model capacity and factual knowledge are acquired during pretraining rather than downstream adaptation, large-scale pretraining constitutes the more fundamental regime for lineage verification. We show that existing fingerprinting methods become *unreliable* in this regime because their post-hoc signatures arise only after substantial optimization. This behavior is inconsistent with the classical Galton criterion for a fingerprint: an identity marker should be intrinsic to an individual object and persistent across its lifetime. We therefore formulate a stronger intrinsic notion of LLM fingerprinting and instantiate it as **SeedPrints**, a method that uses random-initialization biases as persistent, seed-dependent identifiers that exist before any training step is executed. We demonstrate that untrained models exhibit reproducible prediction biases determined by the initialization seed, and that these low-amplitude signals remain statistically detectable throughout training, thereby supporting high-confidence lineage verification. In contrast to prior techniques that fail during early pretraining or degrade under distribution shift, **SeedPrints** remains effective across the full training trajectory, from initialization through large-scale pretraining and downstream adaptation. Experiments on LLaMA-style and Qwen-style architectures establish seed-level distinguishability and support birth-to-lifecycle identity verification. Additional evaluations on large-scale pretraining trajectories and real-world fingerprinting benchmarks confirm robustness under prolonged optimization, domain shift, and parameter-altering transformations.

Collectively, these results show that random initialization itself induces a unique and persistent identity signal in LLMs, yielding a technically grounded “Galtonian” fingerprint. Code will be released upon acceptance.

1 INTRODUCTION

LLM fingerprinting methods have recently been developed to identify, attribute, and trace language models from observable behavioral or structural signatures (Pasquini et al., 2024; Xu et al., 2024; Yoon et al., 2025; Zhang et al., 2024; Zeng et al., 2024). The intended operational role of these methods is to provide model owners with an evidentiary link between a suspicious model and a putative source model, thereby supporting detection of model theft or unauthorized reuse (Yoon et al., 2025; Zhang, 2025).

Much of this literature explicitly adopts the biological-fingerprint analogy introduced by Francis Galton in *Finger Prints* (1892) (Galton, 1892):

“A fingerprint is the pattern formed by friction-ridge skin on the fingertips; this ridge configuration is individually unique and essentially permanent across an individual’s lifetime.”

SeedPrints vs Baselines: OLMo-2-7B Pre-training Checkpoints

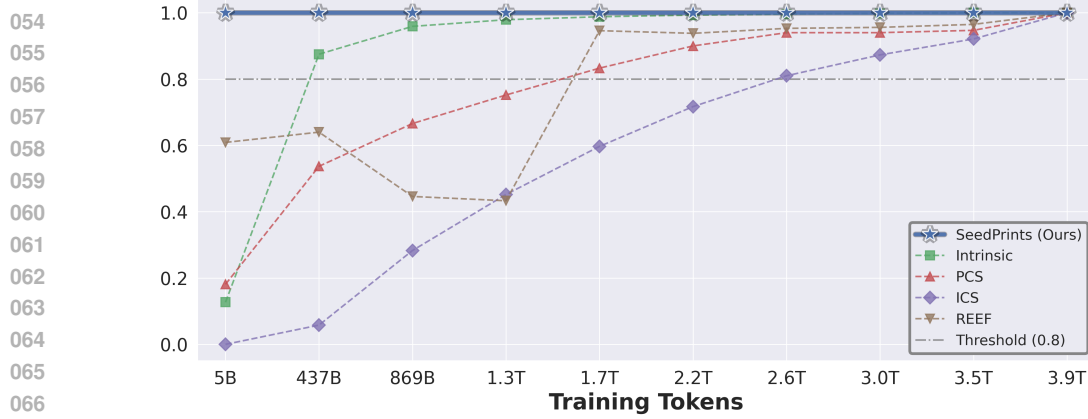


Figure 1: Existing fingerprinting methods do not reliably detect model lineage during early pre-training. We evaluate five methods on OLMo-2-7B checkpoints spanning 5B to 3.9T training tokens, with each checkpoint compared against the final checkpoint. The vertical axis reports the similarity score, where larger values indicate stronger lineage evidence; the dashed line denotes the 0.8 detection threshold. All baseline scores degrade and fall below the threshold for early checkpoints, whereas **SeedPrints** yields perfect detection ($p \ll 0.001$, plotted as $1 - p$) beginning with the first checkpoint.

This analogy imposes two technical requirements on a useful fingerprint: it should be individually discriminative and stable over the relevant lifetime of the object, beginning at the model’s “birth” at random initialization. Most existing fingerprinting approaches do not satisfy this stronger requirement (Pasquini et al., 2024; Xu et al., 2024; Yoon et al., 2025; Zhang et al., 2024; Zeng et al., 2024; Zhang, 2025; Luan et al., 2025; Tsai et al., 2025; Alhazbi et al., 2025). As shown in Figure 1, these methods do not reliably verify lineage during early pretraining. The failure occurs because their signatures are defined only after substantial training has occurred, for example by extracting patterns from trained parameters or generated text. Consequently, the resulting separability does not characterize an initialization-level birthmark of the model; it instead reflects downstream training factors such as data-specific artifacts, optimization dynamics, or hyperparameter configurations. In Section 5.1, for instance, existing baselines frequently fail to recover the correct lineage when the training-data distribution changes substantially. These methods therefore operate as post hoc identifiers rather than **Galtonian fingerprints**—intrinsic identity markers that are present from the beginning and persist thereafter.

In this work, we formalize a stricter notion of LLM fingerprinting as an intrinsic property that exists at *initialization* and remains detectable at *any* later point in training. Our contributions are as follows:

- **Identifying a fundamental limitation of existing methods.** We show that prior fingerprinting approaches do not reliably detect lineage during early pretraining and are susceptible to substantial shifts in the training-data distribution. (Section 5.1)
- **Discovering and leveraging initialization-driven fingerprints.** We show that untrained models contain seed-dependent biases in their internal representations, which constitute a weak but persistent identity signal already present at initialization. (Section 3) We then introduce **SeedPrints**, a procedure for isolating these initialization-born fingerprints while reducing sensitivity to confounders such as data distribution, training duration, and optimization dynamics. (Section 4)
- **Extensive empirical validation.** We demonstrate that the method distinguishes models that differ only in initialization seed, even when the training pipeline and data order are held fixed, and that the signal persists under continual training on diverse datasets. (Section 5.1)
- **Strong performance in realistic settings.** The method detects lineage across large-scale pretraining trajectories (e.g., OLMo-2-7B Stage 1) without early-stage failure. It also remains effective in large-scale fine-tuning settings (e.g., up to 700B tokens) and in practical deployment scenarios from the *LeaFBench* benchmark (Shao et al., 2025), which contains 65 models across 7 mainstream model families and 6 transformations (e.g., instruction tuning, fine-tuning, PEFT, quantization, model merging, and distillation). In these settings, our method matches the strongest near-perfect baseline while substantially outperforming the remaining methods (Sections 5.2 and 5.3).

2 RELATED WORK

LLM protection methods can be organized into two broad technical classes: (i) *watermarking / active fingerprinting* methods that *insert* an identifiable signature into a model or into its generated outputs (Xu et al., 2024; Nasery et al.; Tsai et al., 2025; Sander et al., 2024), and (ii) *passive fingerprinting* methods that *extract* a signature from pre-existing model behavior or structure without modifying the model (Yoon et al., 2025; Zhang et al., 2024; Alhazbi et al., 2025; Pasquini et al., 2024; Suzuki et al., 2025; Zhang, 2025).

Watermarking and Active Fingerprinting. Active approaches deliberately implant verifiable identifiers that can later be queried for ownership verification. Classical instantiations include backdoor attacks (Adi et al., 2018; Li et al., 2019; Zhang et al., 2018), digital signatures, and hash functions (Guo & Potkonjak, 2018; Zhu et al., 2020). For language models, the two dominant forms are *text watermarks*, which bias generation or inject predefined patterns to encode hidden information (Kirchenbauer et al., 2023; Xu et al., 2024; Nasery et al.), and *model weight watermarks*, which embed identifiers into parameters or associate them with secret triggers through fine-tuning (Luan et al., 2025; Li et al., 2023). Although backdoor-style fingerprints are comparatively simple to deploy and may survive moderate fine-tuning (e.g., Dasgupta et al. 2024), such invasive schemes require control over the training process and therefore cannot retroactively mark third-party models.

Passive LLM Fingerprinting. In contrast, passive fingerprinting identifies models by measuring signatures already present in the model, without modifying parameters or outputs. The concrete signature depends on the available access level. Under **white-box access**, signatures are computed from model weights or representations, using quantities such as attention-matrix distributions (Yoon et al., 2025), kernel alignment of internal representations (Zhang et al., 2024), or stable parameter-vector directions. Under **black-box access**, fingerprinting must instead rely on input-output behavior, including crafted queries (Pasquini et al., 2024), distinctive prompt-response pairs (Tsai et al., 2025), stylometric signals (Alhazbi et al., 2025), or iterative prompting-response games (Iourovitski et al., 2024); these behavioral signatures are often less stable under fine-tuning. The common limitation is that these signatures are defined *post hoc*: they measure properties that emerge after training rather than innate, “Galtonian” identifiers present at random initialization. Consequently, *they are not designed for lineage verification during large-scale pretraining*, where post-training signatures may not yet be separable or stable.

3 BIASES ORIGINATING FROM INITIALIZATION PERSIST AFTER TRAINING

This section establishes the empirical observation underlying our method: language models exhibit prediction-bias profiles determined by the random initialization seed, and these profiles remain statistically detectable after training.

Initialization induces seed-specific bias profiles. We evaluate a LLaMA-2-style model initialized with seed 123 on 10,000 random input sequences of length 1,024, with token IDs sampled uniformly from the vocabulary. In Figure 2 (*Left*), we report, for each output token (upper panel, final logits) and each hidden dimension (lower panel, last-layer representations), the empirical frequency with which that coordinate attains the minimum value over the 10,000 random trials. The strongly non-uniform profiles in both panels indicate extreme coordinate-level preference biases. When the same protocol is repeated across initialization seeds, the aggregate bias magnitude remains stable, whereas the particular argmin coordinates vary with the seed (Figure 2, *Lower Right*). These observations are consistent with recent evidence that randomly initialized transformer biases arise from inter-sequence representation contraction, driven by asymmetric nonlinear activations in MLP blocks and amplified by self-attention (Li et al., 2026). Under this mechanism, different initialization seeds select different contraction directions, thereby inducing seed-specific argmin patterns over output dimensions.

Training preserves initialization-born bias Although training changes output magnitudes substantially, it does not fully erase the relative coordinate preferences induced at initialization. We train the seed-123 initialized model for one epoch on OpenWebText (Gokaslan et al., 2019) and evaluate intermediate checkpoints. We restrict attention to the $m = 50$ output dimensions most frequently assigned the minimum value by the initialized model. For each checkpoint, we compute the correlation between checkpoint responses and initialized-model responses on random inputs, restricted to these dimensions. This statistic measures whether the checkpoint and its initialization continue to rank or disfavor the selected dimensions in a consistent input-wise manner. As a negative baseline, we compute the same correlations against independently initialized models (seed 1000). As shown in

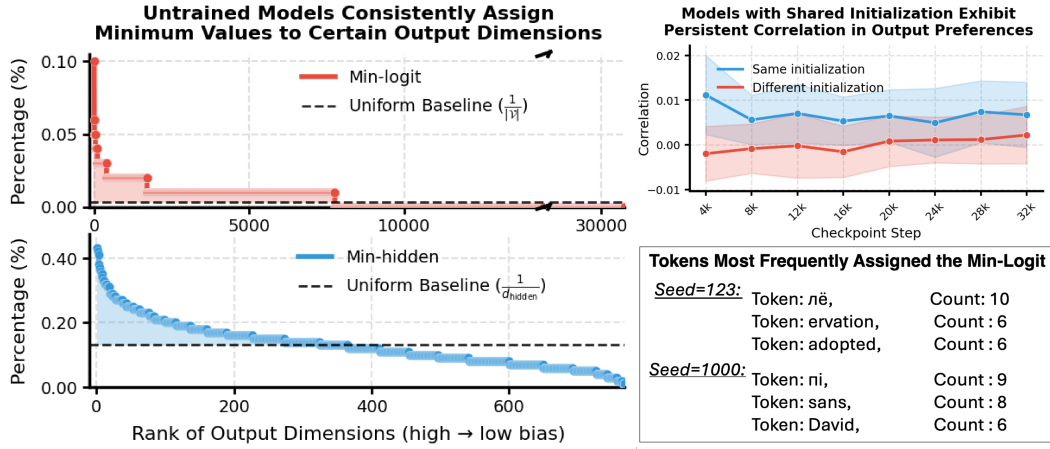


Figure 2: **Initialization-born output bias persists through training.** *Left:* Under fully random inputs, a randomly initialized LLaMA-2-style model produces highly non-uniform output profiles: specific dimensions are systematically disfavored, i.e., they receive the minimum value with high empirical frequency. This extreme bias is visible in both logits (top, red) and final hidden representations (bottom, blue). The dashed line indicates the expected frequency under a uniform distribution. The arrows in the top panel denote a broken x-axis that excludes low-frequency tail ranks. *Upper Right:* During training, model checkpoints retain weak input-wise correlation in output-bias behavior with the initialization from which they descend. The same-lineage correlation distribution is consistently shifted upward relative to correlations with independently initialized base models, indicating a measurable training-persistent initialization signal. *Lower Right:* The high-bias dimensions are seed-dependent; the panel illustrates this dependence using tokens that most frequently attain the minimum logit.

Figure 2 (Upper Right), the absolute correlations are small, but same-initialization correlations are consistently shifted upward relative to the independent baseline and stabilize rather than decaying to zero. This pattern indicates that initialization induces a weak but statistically recoverable bias signal that persists throughout training.

4 ALGORITHM

Section 3 shows that models sharing an initialization lineage retain weak but systematic agreement on a subset of output dimensions, such as dimensions most disfavored by the untrained model under random inputs. The algorithmic objective is therefore to infer, from any two observed models and without access to the true initialization, a set of dimensions carrying the shared bias signal and to test whether their agreement is statistically significant. Algorithm 1 summarizes the complete procedure; the following paragraphs specify each component.

Extract identity dimensions between any two models. Let $X = \{x_i\}_{i=1}^n$, where each $x_i \in \mathbb{R}^{\ell \times d}$ denotes a random sequence of length ℓ . Each random input x_i is instantiated either by uniformly sampling ℓ token IDs from the vocabulary or by directly sampling ℓ independent vectors from a d -dimensional isotropic Gaussian distribution. For any model g , we define its mean response vector as $\bar{g} := \frac{1}{n} \sum_{i=1}^n g(x_i) \in \mathbb{R}^{d_{\text{out}}}$, where d_{out} denotes the output dimensionality. In this definition, $g(x_i)$ may denote either logits or final-layer representations. We define the model’s disfavored dimensions as the bottom- m coordinates of $\bar{g}$:

$$\mathcal{M}_g := \arg \min_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} \bar{g}_j.$$

The mean response vector is used as the bias estimator because it is more stable and noise-robust than per-sample argmin frequency, i.e., counting how often each dimension attains the minimum value as in Figure 2, while also performing well empirically. Appendix C.1 provides a formal comparison.

Given two models f and f' , their *identity dimensions* are defined as the intersection $\mathcal{S} := \mathcal{M}_f \cap \mathcal{M}_{f'}$. If the two models share an initialization lineage, they should independently recover similar disfavored-coordinate sets, producing non-trivial overlap between their bottom- m sets. This intersection criterion functions as mutual verification: it suppresses spurious agreement attributable to architecture or shared training dynamics and retains coordinates more likely to encode shared initialization bias.

When one model is closer to initialization, its disfavored dimensions may also be used directly as an anchor set.

Correlation statistics. After restricting both models to the identity dimensions S , we compare their responses across random inputs.¹ For each $j \in S$, we compute a Kendall–Tau rank correlation:

$$\tau_j = \text{KendallTau}(\{f(x_i)_j\}_{i=1}^n, \{f'(x_i)_j\}_{i=1}^n).$$

This computation produces a set of correlation statistics $T = \{\tau_j\}_{j \in S}$ that represents the distribution of agreement between the two models over identity dimensions. As observed in Section 3, same-lineage models have distributionally larger agreement on these coordinates than unrelated models.

Hypothesis testing via null distribution. Because same-lineage and different-lineage correlation distributions may overlap, thresholding individual correlation values is not a calibrated decision rule. We instead test whether the observed correlations are significantly larger than expected under a null hypothesis of no lineage relation. In principle, the null should be estimated from correlations between independently initialized models, as in Figure 2. Empirically constructing that baseline, however, requires many independent model initializations and pairwise comparisons, which is computationally expensive. Appendix B shows that the null correlation distribution for independently initialized models is well approximated by a Gaussian distribution. This observation motivates a lower-cost simulation alternative: constructing an empirical null distribution by passing Gaussian surrogate matrices through the same pipeline, as summarized in Algorithm 1.

Although the simulation-based null is empirically accurate, it introduces avoidable Monte Carlo randomness and computational overhead: precision is limited by the number of simulation trials, results may vary across runs, and comparison between empirical distributions requires assumptions from a chosen test, such as a one-sided t-test or Mann–Whitney U test. Because the null distribution of Kendall–Tau under weak independence has a closed-form characterization, we replace the simulation procedure with an analytical test, while providing both empirical and analytical implementations in the code repository. Specifically, under the null hypothesis, each Kendall–Tau correlation is approximately zero-mean with known variance σ^2 . By the central limit theorem, as the number of identity dimensions $|S|$ increases, the distribution of $\bar{\tau} = \frac{1}{|S|} \sum_{j \in S} \tau_j$ is well approximated by $\bar{\tau} \sim \mathcal{N}\left(0, \frac{\sigma^2}{|S|}\right)$. Row-wise softmax normalization introduces weak dependencies across dimensions, but the use of a relatively high temperature ($T = 10$) mitigates these dependencies sufficiently for the independence approximation to be accurate in practice. Appendix B empirically validates that the resulting Gaussian approximation closely matches the full-pipeline null distribution. We therefore compute the one-sided z-score $z = \frac{\bar{\tau}}{\sigma/\sqrt{|S|}}$ and the corresponding p-value $p = 1 - \Phi(z)$, where Φ denotes the standard normal cumulative distribution function. The null hypothesis is rejected, and shared lineage is declared, when $p < 0.01$.

Practical instantiation. Model outputs may be instantiated as either logits or hidden representations. Both choices are compatible with the formulation, but logits require substantially more random inputs to estimate bias patterns reliably as vocabulary size increases, which induces significant computational overhead (see Appendix C.2 for the theoretical analysis). For efficiency and statistical stability, we therefore use final-layer hidden states as the default output representation. The resulting p-value is a calibrated lineage test rather than a heuristic similarity threshold.

5 EXPERIMENTS

This section is organized around two evaluation regimes. In Section 5.1, we test whether the proposed method satisfies the requirements of a genuine fingerprint: (i) it supports *birth verification at the seed level*, and (ii) it remains *verifiable across the entire training lifecycle*. Existing baselines, by contrast, do not support reliable verification during early pretraining and primarily exploit data-dependent signatures that can fail under substantial distribution shift. Unless otherwise stated, we report results using the analytical null with *hidden state* outputs; Appendix D.2 additionally reports empirical-null variants with both *logit* and *hidden state* outputs, evaluated using the one-sided t-test and Mann–Whitney U test. In Section 5.3, we evaluate all methods in practical infringement scenarios using

¹Before computing the correlations, we apply a row-wise softmax normalization to each sample’s output vector. This normalization removes scale differences across models and dimensions while preserving the relative order information used by rank-based statistics such as Kendall–Tau. A relatively high temperature is used to avoid degenerate one-hot behavior.

Algorithm 1 Distribution Correlation Test on Identity Dimensions

Require: base model f , suspicious model f' ; random inputs $X = \{x_i\}_{i=1}^n$; fingerprint size m ; significance level α

► **Step 1: Localize biased dimensions**

- 1: Compute average outputs $\bar{f}, \bar{f}'$ over X
- 2: $\mathcal{M}(f), \mathcal{M}(f') \leftarrow$ bottom- m dimensions of $\bar{f}, \bar{f}'$
- 3: $\mathcal{S} \leftarrow \mathcal{M}(f) \cap \mathcal{M}(f')$ (identity dimensions)

► **Step 2: Form correlation distribution**

- 4: Normalize $f(x_i)_{\mathcal{S}}, f'(x_i)_{\mathcal{S}}$ for each $x_i \in X$ across identity dimension
- 5: **for** each $s_j \in \mathcal{S}$ **do**
- 6: $\tau_j \leftarrow \text{KendallTau}(\{f(x_i)_{s_j}\}_{i=1}^n, \{f'(x_i)_{s_j}\}_{i=1}^n)$ (correlation in input-wise preferences)
- 7: **end for**
- 8: $\mathcal{T} \leftarrow \{\tau_j\}_{j=1}^{|\mathcal{S}|}$

► **Step 3: Hypothesis test against null**

- 9: Construct $\mathcal{T}_{\text{null}}$ by applying the same pipeline to two Gaussian matrices $Y^{(1)}, Y^{(2)} \sim \mathcal{N}(0, I)^{N \times d_{\text{out}}}$
- 10: Test $H_0 : \mathcal{T} = \mathcal{T}_{\text{null}}$ vs. $H_1 : \mathcal{T} > \mathcal{T}_{\text{null}}$
- 11: **Return** SameLineage $\leftarrow \mathbf{1}(p\text{-value} < \alpha)$

Table 1: Comparison of fingerprint behavior between models initialized with different seeds. Table 2: Trained models preserve the fingerprint behavior of their initialization (p -value < 0.01).

Model Pairs	p -value (> 0.01)	Model Pairs	SeedPrints	Baselines			
			p -value	Intrinsic	REEF	PCS	ICS
s_{42} VS. s_{2000}	0.116	s_{42}^{init} vs. s_{42}^{base}	3.37e-26✓	-0.021×	0.375×	0.580×	0.196×
s_{123} VS. s_{42}	1.000	s_{123}^{init} vs. s_{123}^{base}	3.16e-33✓	0.149×	0.369×	0.581×	0.188×
s_{1000} VS. s_{123}	0.327	s_{1000}^{init} vs. s_{1000}^{base}	1.12e-21✓	-0.252×	0.381×	0.581×	0.188×
s_{2000} VS. s_{1000}	1.000	s_{2000}^{init} vs. s_{2000}^{base}	3.70e-31✓	-0.337×	0.331×	0.581×	0.188×

LeafBench (Shao et al., 2025). This benchmark contains 65 distinct model instances spanning 6 representative post-development transformations, allowing us to assess whether SeedPrints remains competitive with post-training baselines while retaining robustness to deployment-time parameter changes.

Baselines We evaluate four primary passive fingerprinting baselines based on weights or representations. *Intrinsic fingerprint* (Yoon et al., 2025) (also denoted *PDF* in some prior work) compares models using the similarity of layerwise standard-deviation profiles of attention parameters. *REEF* (Zhang et al., 2024) computes centered-kernel-alignment (CKA) similarity between feature representations produced by two models on shared samples. *PCS* and *ICS* (Zeng et al., 2024) (collectively referred to as *HuRef* in some prior work) are weight-similarity methods: PCS flattens all parameters and computes cosine similarity, whereas ICS constructs invariant weight terms and computes cosine similarity over those invariants. In Section 5.3, we also include the weaker gradient-based fingerprint *Gradient* (Shao et al., 2025). Following Zhang et al. (2024), binary decisions for similarity-based methods use a **0.8 similarity threshold**.

Throughout the experiment tables, cell colors encode ground-truth lineage: **green** denotes same-source model pairs, and **red** denotes different-source pairs. For example, s_{42}^{init} vs. s_{42}^{base} compares an initialization with seed 42 against its continued-pretrained descendant and is therefore a same-lineage pair. Conversely, s_{2000}^{init} vs. s_{42}^{base} compares a seed-2000 initialization against a model trained from seed 42 and is therefore a different-lineage pair. The symbol ✓ indicates a correct decision, whereas × indicates an error.

5.1 BIRTH-TO-LIFECYCLE “BIOMETRIC” FINGERPRINTING

We train 12-layer, 12-head LLaMA-style models (Touvron et al., 2023) with RoPE (Su et al., 2021), as well as Qwen-style models (Team, 2024), from scratch. The main text reports LLaMA-style results, while Appendix D reports the corresponding Qwen-style experiments. The empirical conclusions are consistent across the two architectural families.

Different initialization seeds produce distinct fingerprints Table 1 reports p -values from correlation tests between model pairs initialized with different random seeds (42, 123, 1000, and 2000). All reported p -values remain > 0.01 , indicating that the test distinguishes models whose only intended

Table 3: Identical datasets and training order do not make fingerprint behavior identical across different initializations.

Model Pair	p -value (> 0.01)
s_{42}^{init} vs. s_{123}^{base}	0.573
s_{123}^{init} vs. s_{2000}^{base}	0.724
s_{2000}^{init} vs. s_{1000}^{base}	0.109
s_{1000}^{init} vs. s_{42}^{base}	0.883

Table 4: Fingerprint persistence under continual training on diverse datasets, using the seed-1000 base model pretrained on openwebtext.

Setting	SeedPrints	Baselines			
Continual corpus (seed)	p -value	Intrinsic	REEF	PCS	ICS
TinyStories (1000)	≈ 0 ✓	1.000✓	0.759×	0.999✓	0.996✓
TinyStories (123)	1.000✓	0.950×	0.658✓	0.332✓	0.012✓
the_stack (1000)	≈ 0 ✓	0.489×	0.557×	0.585×	0.123×
the_stack (123)	1.000✓	0.445✓	0.580✓	0.301✓	0.026✓

difference is the initialization seed. This result establishes that different seeds induce distinct fingerprint behaviors and that these behaviors are separable at model “birth.”

Training preserves the initialization fingerprint. Table 2 compares each initialized model s_i^{init} with its descendant s_i^{base} trained on OpenWebText (Gokaslan et al., 2019) (≈ 10 B tokens). For every seed–model pair, the p -value is < 0.01 , indicating that the trained descendant retains a bias profile strongly correlated with its initialization and therefore shares the same lineage. Thus, the trained model inherits the initialization fingerprint. The baseline methods fail uniformly to separate seeds in this setting, suggesting that their discriminative signals arise from training-induced artifacts rather than initialization-level identity.

Identical data and order do not make fingerprints converge In Table 3, all four “suspicious” models s_i^{base} for $i \in \{42, 123, 100, 2000\}$ are trained on *exactly the same corpus (OpenWebText) and in the same data order*; the training seed is fixed to control the data order. The only experimental variable is the initialization seed i . Across all cross-seed pairs, p -values remain consistently > 0.01 , in contrast to the near-zero values in Table 2. Therefore, seed-specific fingerprints do not collapse under identical data exposure or curriculum.

Continual training on diverse datasets does not confound the fingerprint The preceding experiments are designed to establish the biometric property of SeedPrints: the method can distinguish initialization-level identity. From a copyright-auditing perspective, the inability of prior work to distinguish different initialization seeds is not necessarily a defect, because initialization seeds themselves have no clear copyright status. The relevant fragility is instead that prior attribution signals can be driven by *data distribution*, thereby *failing to recognize lineage when continued training introduces a substantial distribution shift*.

In Table 4, we continue training a seed-1000 base model pretrained on OpenWebText (Gokaslan et al., 2019) on two highly different datasets: TinyStories (Eldan & Li, 2023), consisting of synthetic children’s stories, and The Stack (Kocetkov et al., 2022), consisting of permissively licensed GitHub code. We compare (i) true descendants trained from the base model with (ii) *distractors* derived from a different base model with seed 123. Because the baseline methods are not sensitive to initialization differences, the distractor base is pretrained on OpenWebText using a different data order and is then continually trained on the *same* target corpus as the true descendant. The evaluation question is whether each attribution method identifies the descendant that actually shares lineage with the source base.

All prior baselines fail in the code-domain setting (The Stack), misclassifying true descendants as distractors. This failure indicates that their scores largely track domain similarity rather than lineage identity: TinyStories is closer to the OpenWebText pretraining distribution, whereas The Stack induces a much larger domain shift, which is sufficient to bypass detection. SeedPrints correctly attributes lineage for both corpora. The fingerprint therefore is not a proxy for data distribution; it remains detectable under substantial domain shift and after additional pretraining.

5.2 ALL-STAGE VERIFIABLE FINGERPRINTS

Controlled pretraining trajectory. Can SeedPrints identify descendants throughout a training trajectory? We first test this question in a controlled OpenWebText pretraining experiment in which the initialization seed is known. Lineage is evaluated at intermediate checkpoints, and the method detects the correct lineage at all stages (Appendix D.2.1, Figure 5).

Table 5: Fingerprinting results under large-scale finetuning. Each row compares a target model against LLaMA-2-7B. **SeedPrints** reports the p -value from the correlation test (< 0.01 indicates statistically significant lineage evidence). The four baselines report similarity scores (threshold = 0.8, higher = better).

Model	# Tokens	SeedPrints p	Intrinsic $\uparrow$	REEF ($\uparrow$)	PCS ($\uparrow$)	ICS ($\uparrow$)
Llama-2-finance-7B (Heenan, 2023)	5M	10^{-41955} ✓	1.0000 ✓	0.9950 ✓	0.9979 ✓	0.9952 ✓
Vicuna-1.5-7B (Chiang et al., 2023)	370M	$10^{-103043}$ ✓	1.0000 ✓	0.9985 ✓	0.9985 ✓	0.9949 ✓
Wizardmath-7B (Luo et al., 2023)	1.8B	$10^{-180550}$ ✓	1.0000 ✓	0.9979 ✓	1.0000 ✓	0.9994 ✓
Meditron-7B (Chen et al., 2023)	48B	10^{-42566} ✓	0.9990 ✓	0.9978 ✓	1.0000 ✓	0.9817 ✓
CodeLlama-7B (Meta AI, 2024)	500B	10^{-3552} ✓	0.9480 ✓	0.9947 ✓	0.6863 ✗	0.3369 ✗
Llemma-7B (Azerbayev et al., 2023)	700B	10^{-5136} ✓	0.9470 ✗	0.9984 ✓	0.6682 ✗	0.2905 ✗

Large-scale pretraining. To evaluate lineage under realistic foundation-model training, we analyze ten checkpoints from the OLMo-2-7B (OLMo et al., 2024) Stage-1 run (5B $\rightarrow$ 3.9T tokens). The final checkpoint is treated as the target model, and all preceding checkpoints are treated as candidate ancestors. Unlike the controlled OpenWebText setting, this trajectory includes heterogeneous corpora, substantially longer optimization, and large-scale representation drift. For comparability with similarity-based baselines, we plot $1 - p$ so that larger values indicate stronger lineage evidence. All p -values are $\ll 0.01$ and numerically close to zero.

As shown in Figure 1, **all existing baselines fail to verify lineage during early pretraining**: within the first **trillion** tokens, ICS is approximately 0, PCS is below 0.3, and REEF is below 0.6. In contrast, SeedPrints remains verifiable throughout the trajectory and its evidence strengthens with training. These results expose a limitation in common evaluation protocols: **most prior work evaluates lineage only after fine-tuning, when detection is substantially easier**. Large-scale pretraining can itself reshape representations and strengthen post-training lineage signals, which may create a misleading impression of safety if early checkpoints are not evaluated. Lineage verification should therefore emphasize early pretraining stages, where unauthorized reuse is most difficult to detect, rather than relying exclusively on late-stage checkpoints.

Large-scale finetuning. We further compare SeedPrints with existing baselines under standard fine-tuning-stage evaluations. Specifically, we test suspect models fine-tuned from the Llama-2-7b base model using data volumes ranging from 5 million to 700 billion tokens. The suspect set includes downstream variants such as Llama-2-finance-7b (Heenan, 2023), Vicuna-1.5-7b (Chiang et al., 2023), WizardMath-7b (Luo et al., 2023), Chinese-LLaMA-2-7b (Chen et al., 2023), Code-Llama-7b (Meta AI, 2024), and Llemma-7b (Azerbayev et al., 2023). Their fine-tuning data volumes are 5M, 370M, 1.8B, 13B, 500B, and 700B tokens, respectively. As reported in Table 5, SeedPrints maintains $p < 0.01$ in every setting.

5.3 ROBUSTNESS UNDER REALISTIC DEPLOYMENTS

Real-world deployment pipelines often apply *parameter-altering* adaptations to foundation models, including fine-tuning, PEFT, and quantization; such transformations can attenuate or distort fingerprint signals. SeedPrints is designed to remain effective under these transformations. To evaluate this property under realistic auditing conditions, we use *LeaFBench* (Shao et al., 2025), a benchmark for language-model copyright auditing.

Across Source Models. We consider two source-model types for auditing: *Pre-Trained (PT)* and *Instruction-Tuned (IT)*. For each source type, the task is to distinguish *derivative* models, obtained by post-training adaptations, from *independent* models trained without the source weights. This protocol matches common auditing scenarios in which a claimant must evaluate lineage relative to either a PT base model or its IT variant. The evaluation contains **65** models in total; Appendix E lists the full set of model names and HuggingFace repositories.

Metrics. Detection performance is measured using (i) Area Under the ROC Curve (AUC) and (ii) the Kolmogorov–Smirnov (KS) statistic (Berger & Zhou, 2014), which quantifies the maximum separation between derivative-model and independent-model score distributions.

SeedPrints outputs per-model-pair p -values from statistical tests, whereas the baseline methods output similarity scores. These quantities have different semantics: similarity scores are linear measures for which score differences are interpreted proportionally, while p -values are tail probabilities under

the null hypothesis and therefore are not linear-scale similarity measures. To compute AUC in the same format as similarity-based baselines, we convert p-values to scores using $s = 1 - p$. This conversion is not fully aligned with the statistical interpretation of p-values and may be suboptimal for SeedPrints, because small p-values vary on an exponential scale whereas threshold sweeping is linear. Nevertheless, the converted scores yield comparable or superior empirical performance. For evaluation, we (i) compute AUC for all methods by sweeping thresholds over scores s , and (ii) report the KS statistic as a threshold-free measure of distributional separability. This protocol preserves compatibility with baseline pipelines while retaining the calibrated statistical interpretation of SeedPrints. We do not report Manhattan distance because absolute distances between p-values are not meaningful.

In many practical auditing settings, an optimal similarity threshold cannot be calibrated reliably. Although AUC is reported for comparability, SeedPrints differs structurally from prior methods because it provides a direct hypothesis test. **Given any pair of models, our approach outputs a p-value that enables a definitive decision on whether they share the same lineage, without relying on tunable thresholds**, which makes the decision rule operational in practice.

Parameter-Altering Techniques. To evaluate robustness to deployment transformations, we test models produced by (1) Instruction tuning (Instruct), (2) General-purpose fine-tuning (Finetune), (3) Parameter-efficient fine-tuning (PEFT), (4) Quantization, (5) Model merging (Merge), and (6) Distillation.

5.3.1 FAMILY-WISE EFFECTIVENESS

The benchmark contains seven model families. For each family, we treat the pretrained base model (PT) and the instruction-tuned model (IT) as separate sources and compare each source against all other models in the 65-model pool. Table 6 reports the results. TinyLlama-1.1B families are omitted from Table 6 because they are trivial for all methods. Each family-specific column corresponds to AUC computed over tests within that family using a family-calibrated threshold, whereas the overall score is computed over all test pairs using a global threshold. Thus, the overall score is not the arithmetic mean of the family-specific scores. The three strongest methods—*Intrinsic*, *ICS*, and *SeedPrints*—are nearly saturated across alterations, with overall AUC values of 0.994, 0.994, and 0.992, respectively, and KS values of 0.989, 0.943, and 0.986. For these methods, most family-wise scores reach the maximum. In contrast, *REEF* and *Gradient* perform substantially worse. These results indicate that, although SeedPrints is designed to capture intrinsic seed-level fingerprints, it also functions as a biometric-like fingerprint for persistent identity tracking across diverse model families.

5.3.2 ALTERATION-WISE ROBUSTNESS

Across alteration types, *Instruct* is consistently the easiest condition: even the weakest baseline, *Gradient*, reaches AUC 0.895. *Finetune* and *Merge* are more challenging for all methods; for *Finetune*, no method attains perfect performance. The effects of *PEFT*, *Quantization*, and *Distillation* are heterogeneous. *REEF* drops substantially under *Quantization* ($0.996 \rightarrow 0.871$) and *Distillation* ($0.996 \rightarrow 0.858$), while remaining robust under *PEFT* (0.994). Conversely, *PEFT* and *Quantization* are more difficult for *Gradient* ($0.895 \rightarrow 0.776/0.756$). The other three stronger methods remain robust across transformations, with all scores > 0.9 . SeedPrints decreases modestly to 0.959 under *Merge*, corresponding to one failed merge model out of seventeen. Although this is a single misclassification, it suggests that weight-space interpolation with unrelated model weights can reduce the initialization signal below the detection threshold. Averaged over all six alteration types, *Intrinsic* obtains the highest overall AUC, followed closely by *ICS* and *SeedPrints*.

5.4 ABLATION STUDY ON HYPERPARAMETERS

SeedPrints has three hyperparameters: the number of random input sequences n , the length of each random sequence l , and the fingerprint size m . The first two determine the total random-input budget, whereas m specifies the number of low-mean coordinates retained for testing. In the main experiments in Sections 5.2 and 5.3, we set $n=2000$, $l=1024$, and $m=0.1d_{\text{out}}$, where d_{out} is the hidden-state dimensionality; this parameterization scales the fingerprint size to different open-source model dimensions. In this ablation, each hyperparameter is varied while the others are fixed at their default values (i.e., $n = 2000$, $m = 400$, $l = 768$), and Table 8 reports empirical accuracy and false positive rate at significance level $\alpha = 0.01$.

Table 6: Performance comparison of LLM fingerprinting methods across different source models. “PT” and “IT” refer to using the pre-trained models and instruction-tuned models as source models, respectively. Per-family breakdown is shown for the 6 families that have both PT and IT models. Note that the “Overall” scores are computed over all test model pairs, not averaged across families.

Method	Metric	Qwen2.5-7B		Qwen2.5-14B		Llama3.1-8B		Mistral-7B		Gemma2-2B		Llama2-7B		Overall
		PT	IT	PT	IT	PT	IT	PT	IT	PT	IT	PT	IT	
REEF	AUC $\uparrow$	0.796	0.862	0.947	0.913	1.000	0.999	0.997	0.997	0.963	0.968	0.750	0.706	0.915
	KS Statistic $\uparrow$	0.492	0.633	0.875	0.735	0.987	0.987	0.975	0.980	0.950	0.837	0.750	0.694	0.739
Gradient	AUC $\uparrow$	0.657	0.743	0.763	0.789	0.897	0.904	0.872	0.682	1.000	1.000	0.650	0.768	0.801
	KS Statistic $\uparrow$	0.354	0.423	0.500	0.563	0.646	0.705	0.725	0.500	1.000	1.000	0.275	0.479	0.508
ICS	AUC $\uparrow$	1.000	1.000	0.997	0.997	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.994
	KS Statistic $\uparrow$	1.000	1.000	0.975	0.979	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.943
Intrinsic	AUC $\uparrow$	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.994
	KS Statistic $\uparrow$	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.989
SeedPrints	AUC $\uparrow$	0.992	0.994	0.988	0.990	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.992
	KS Statistic $\uparrow$	0.985	0.987	0.975	0.980	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.986

Table 7: Performance of LLM fingerprinting methods across parameter-altering transformation types.

Method	Metric	Instruct	Finetune	PEFT	Quantization	Merge	Distillation	Overall Avg
REEF	AUC $\uparrow$	0.996	0.872	0.994	0.871	0.954	0.858	0.924
	KS Statistic $\uparrow$	0.949	0.637	0.940	0.812	0.851	0.615	0.801
Gradient	AUC $\uparrow$	0.895	0.729	0.776	0.756	0.832	0.887	0.812
	KS Statistic $\uparrow$	0.756	0.441	0.469	0.586	0.519	0.679	0.575
ICS	AUC $\uparrow$	1.000	0.987	1.000	1.000	0.981	1.000	0.995
	KS Statistic $\uparrow$	1.000	0.925	1.000	1.000	0.941	1.000	0.978
Intrinsic	AUC $\uparrow$	1.000	0.980	1.000	1.000	1.000	1.000	0.997
	KS Statistic $\uparrow$	1.000	0.971	1.000	1.000	1.000	1.000	0.995
SeedPrints	AUC $\uparrow$	1.000	0.995	0.990	1.000	0.959	1.000	0.991
	KS Statistic $\uparrow$	1.000	0.990	0.980	1.000	0.941	1.000	0.985

Table 8: Hyperparameter ablation at significance level $\alpha = 0.01$.

Ablation on n	200	500	2000
ACC	0.7368	0.7778	0.9375
Empirical FPR	0	0	0
Ablation on l	512	768	1024
ACC	0.9048	0.9375	1.0000
Empirical FPR	0.04	0	0
Ablation on m	200	400	800
ACC	0.9375	0.9375	0.875
Empirical FPR	0.04	0	0.1463

Increasing n and l consistently improves identification performance. This trend is predicted by the theoretical analysis in Section C.2: to recover the population-level bias accurately, the required number of queries must satisfy $n = \Omega\left(\frac{\log d_{\text{out}}}{\gamma^2}\right)$, where γ is the bias margin separating biased coordinates from the remaining dimensions. Thus, each target output dimension requires a minimum sample budget for reliable detection, which explains the degraded performance at smaller n . The margin γ increases with l because longer sequences induce accumulation effects and therefore stronger coordinate separation. Under a fixed and insufficient number of sequences, increasing l reduces the query budget required for detection and improves performance.

The effect of m is non-monotonic. If m is too small, the selected coordinate set is unstable because the empirical observations are insufficient to define a reliable distribution. If m is too large, many irrelevant coordinates are included, which injects noise and reduces γ ; compensating for this reduction would require quadratically more samples. Empirically, m values in the range 200–400 perform well across model sizes, consistent with the heuristic $m \approx 0.1d_{\text{out}}$ for typical $d_{\text{out}} \in [3000, 5000]$.

6 CONCLUSION

This work introduced *SeedPrints*, an intrinsic LLM fingerprinting framework that traces model lineage to the random initialization event. We showed that initialization induces a persistent “Galtonian” fingerprint in neural language models: untrained models already exhibit stable, seed-dependent token-preference and representation-bias patterns, and these biases remain statistically measurable throughout training. Building on this observation, we defined a distribution-correlation test over *identity dimensions* that detects shared lineage with calibrated significance. Across LLaMA- and Qwen-style model families and realistic deployment transformations (e.g., continued pretraining

on divergent corpora, instruction tuning, PEFT, quantization, merging, and distillation), SeedPrints supports “birth-to-lifecycle” verification and remains robust under domain shift, providing a practical mechanism for provenance and copyright auditing.

Limitations and Future Directions This work targets fingerprint signals rooted in initialization-stage bias. The proposed method requires access to model outputs, such as logits or hidden states, and may not be applicable when access is restricted to limited black-box APIs. Nevertheless, the search for *true fingerprints* for LLMs remains technically underdeveloped: most existing approaches, particularly black-box methods, depend on signals that emerge only in later training stages or on alignment-induced artifacts, and therefore do not robustly protect pretrained or early-stage models. This limitation makes reliable all-stage fingerprints a pressing research objective. Additional discussion appears in Section [A.2](#).

REFERENCES

- Yossi Adi, Carsten Baum, Moustapha Cisse, Benny Pinkas, and Joseph Keshet. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX security symposium (USENIX Security 18)*, pp. 1615–1631, 2018.
- Saeif Alhazbi, Ahmed Hussain, Gabriele Oligeri, and Panos Papadimitratos. Llms have rhythm: Fingerprinting large language models using inter-token times and network traffic analysis. *IEEE Open Journal of the Communications Society*, 2025.
- Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631*, 2023.
- Vance W Berger and YanYan Zhou. Kolmogorov–smirnov test: Overview. *Wiley statsref: Statistics reference online*, 2014.
- Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramer. Membership inference attacks from first principles. In *2022 IEEE symposium on security and privacy (SP)*, pp. 1897–1914. IEEE, 2022.
- Zeming Chen, Alejandro Hernández-Cano, Angelika Romanou, Antoine Bonnet, Kyle Matoba, Francesco Salvi, Matteo Pagliardini, Simin Fan, Andreas Köpf, Amirkeivan Mohtashami, Alexandre Sallinen, Alireza Sakhaeirad, Vinitra Swamy, Igor Krawczuk, Deniz Bayazit, Axel Marmet, Syrielle Montariol, Mary-Anne Hartley, Martin Jaggi, and Antoine Bosselut. Meditron-70b: Scaling medical pretraining for large language models, 2023.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Agnibh Dasgupta, Abdullah Tanvir, and Xin Zhong. Watermarking language models through language models. *arXiv preprint arXiv:2411.05091*, 2024.
- Ronen Eldan and Yuanzhi Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- Francis Galton. *Finger prints*. Number 57490-57492. Cosimo Classics, 1892.
- Aaron Gokaslan, Vanya Cohen, Ellie Pavlick, and Stefanie Tellex. Openwebtext corpus. <http://Skyllion007.github.io/OpenWebTextCorpus>, 2019.
- Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
- Jia Guo and Miodrag Potkonjak. Watermarking deep neural networks for embedded systems. In *2018 IEEE/ACM international conference on computer-aided design (ICCAD)*, pp. 1–8. IEEE, 2018.
- Collin Heenan. Llama2-7b-finance (hugging face model). <https://huggingface.co/cxllin/Llama2-7b-Finance>, 2023. Fine-tuned from NousResearch/Llama-2-7b-hf; MIT License; accessed 2025-09-02.
- Dmitri Iourovitski, Sanat Sharma, and Rakshak Talwar. Hide and seek: Fingerprinting large language models with evolutionary learning. *arXiv preprint arXiv:2408.02871*, 2024.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *International Conference on Machine Learning*, pp. 17061–17084. PMLR, 2023.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 tb of permissively licensed source code. *Transactions on Machine Learning Research (TMLR), Preprint*, 2022.

- Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. *Advances in neural information processing systems*, 32, 2019.
- Peixuan Li, Pengzhou Cheng, Fangqi Li, Wei Du, Haodong Zhao, and Gongshen Liu. Plmmark: a secure and robust black-box watermarking framework for pre-trained language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 14991–14999, 2023.
- Siquan Li, Yao Tong, Haonan Wang, and Tianyang Hu. Transformers are born biased: Structural inductive biases at random initialization and their practical consequences. *arXiv preprint arXiv:2602.05927*, 2026.
- Zheng Li, Chengyu Hu, Yang Zhang, and Shanqing Guo. How to prove your model belongs to you: A blind-watermark based framework to protect intellectual property of dnn. In *Proceedings of the 35th annual computer security applications conference*, pp. 126–137, 2019.
- Xiaokun Luan, Zeming Wei, Yihao Zhang, and Meng Sun. Robust and efficient watermarking of large language models using error correction codes. *Proceedings on Privacy Enhancing Technologies*, 2025.
- Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*, 2023.
- Meta AI. Codellama-7b-hf: Code llama base 7b model (hugging face). <https://huggingface.co/codellama/CodeLlama-7b-hf>, 2024. Base 7 billion-parameter Code Llama model for code synthesis and understanding; trained between January and July 2023; licensed under Meta Llama 2 license; accessed 2025-09-02.
- Anshul Nasery, Jonathan Hayase, Creston Brooks, Peiyao Sheng, Himanshu Tyagi, Pramod Viswanath, and Sewoong Oh. Scalable fingerprinting of large language models. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*.
- Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 olmo 2 furious, 2024. URL <https://arxiv.org/abs/2501.00656>.
- Dario Pasquini, Evgenios M Kornaropoulos, and Giuseppe Ateniese. Llmmap: Fingerprinting for large language models. *arXiv preprint arXiv:2407.15847*, 2024.
- Tom Sander, Pierre Fernandez, Alain Durmus, Matthijs Douze, and Teddy Furon. Watermarking makes language models radioactive. *Advances in Neural Information Processing Systems*, 37: 21079–21113, 2024.
- Shuo Shao, Yiming Li, Yu He, Hongwei Yao, Wenyan Yang, Dacheng Tao, and Zhan Qin. Sok: Large language model copyright auditing via fingerprinting. *arXiv preprint arXiv:2508.19843*, 2025.
- Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 115–124. Association for Computational Linguistics, 2021.
- Teppe Suzuki, Ryokan Ri, and Sho Takase. Natural fingerprints of large language models. *arXiv preprint arXiv:2504.14871*, 2025.
- Qwen Team. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2, 2024.

-
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Yun-Yun Tsai, Chuan Guo, Junfeng Yang, and Laurens van der Maaten. Rofl: Robust fingerprinting of language models. *arXiv preprint arXiv:2505.12682*, 2025.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- Jiashu Xu, Fei Wang, Mingyu Derek Ma, Pang Wei Koh, Chaowei Xiao, and Muhao Chen. Instructional fingerprinting of large language models. *arXiv preprint arXiv:2401.12255*, 2024.
- Do-hyeon Yoon, Minsoo Chun, Thomas Allen, Hans Müller, Min Wang, and Rajesh Sharma. Intrinsic fingerprint of llms: Continue training is not all you need to steal a model! *arXiv preprint arXiv:2507.03014*, 2025.
- Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. Huref: Human-readable fingerprint for large language models. *Advances in Neural Information Processing Systems*, 37:126332–126362, 2024.
- Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph Stoecklin, Heqing Huang, and Ian Molloy. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia conference on computer and communications security*, pp. 159–172, 2018.
- Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. Reef: Representation encoding fingerprints for large language models. *arXiv preprint arXiv:2410.14273*, 2024.
- Ruichong Zhang. Matrix-driven instant review: Confident detection and reconstruction of llm plagiarism on pc. *arXiv preprint arXiv:2508.06309*, 2025.
- Renjie Zhu, Xinpeng Zhang, Mengte Shi, and Zhenjun Tang. Secure neural network watermarking protocol against forging attack. *EURASIP Journal on Image and Video Processing*, 2020(1):37, 2020.

756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809

Appendix Contents

A	LLM Usage and Limitation Discussions	16
A.1	LLM Usage	16
A.2	Limitations and Future Directions	16
B	Validation of the Empirical and Analytical Null Distribution	16
B.1	Empirical True Null from Independently Initialized Models	16
B.2	Gaussian Surrogate as a Simulation-Based Approximation	16
B.3	Analytical Approximation under Weak Dependence	17
C	Theoretical Properties of Our Methods	18
C.1	Stability Analysis of Mean-Based Identity Extraction vs. Per-Sample Argmin Voting	19
C.1.1	Stability of Sample-Mean-Based Extraction	19
C.1.2	Instability of Per-Sample Argmin Voting	20
C.2	Influence of Hyperparameters	22
D	More Experiment Details	23
D.1	Implementation Details and Licenses	23
D.2	Complementary SeedPrints Results using simulation-based null	23
D.2.1	Birth-to-Lifecycle “Biometric” Fingerprinting	23
D.2.2	All-stage verifiable fingerprints	25
D.3	Cross-Size Fingerprint Within the Qwen-2.5 series	26
E	Model Catalog and Decoding Settings	26
E.1	Model list by family	27
E.1.1	Qwen-2.5-7B	27
E.1.2	Qwen2.5-14B	27
E.1.3	Llama-3.1-8B	27
E.1.4	Mistral-7B-v0.3	28
E.1.5	Gemma-2-2B	28
E.1.6	Llama-2-7B	28
E.1.7	TinyLlama-1.1B	28
E.2	Sample Results	29

A LLM USAGE AND LIMITATION DISCUSSIONS

A.1 LLM USAGE

We use AI assistants, i.e., ChatGPT and Gemini, for writing and formatting support, including grammar checks, caption clarity, and other surface-level editorial revisions. For programming-related tasks, we occasionally use GitHub Copilot and Claude as coding assistants for code completion and debugging suggestions.

A.2 LIMITATIONS AND FUTURE DIRECTIONS

Although this work identifies seed-born, training-persistent biases that can function as fingerprint signals, the proposed method remains within the white-box access paradigm. It may not apply when only restricted API access is available, for example when responses are deterministic, safety-filtered, or otherwise insufficiently informative. Nevertheless, the development of *true fingerprints* for LLMs remains at an early technical stage, even under white-box access. Most existing fingerprinting approaches depend on training-induced signatures or alignment behaviors and therefore cannot robustly protect pretrained or early-stage models. By analogy to human fingerprints, which remain stable across age and environment, a true LLM fingerprint should persist across continued training and deployment contexts rather than encode incidental training artifacts. Achieving this property, particularly under black-box access where a substantial performance gap remains relative to white-box methods, is an important open problem requiring further work. We hope this study motivates more principled and robust fingerprinting frameworks for large models.

B VALIDATION OF THE EMPIRICAL AND ANALYTICAL NULL DISTRIBUTION

The hypothesis test in Section 4 evaluates whether two models share lineage by comparing the distribution of their correlation statistics on random inputs against a null distribution. This null distribution represents the correlation statistics expected from two *independent models evaluated on random inputs*.

We consider two approximations to this null distribution: (i) an empirical simulation-based null obtained by applying the full pipeline to Gaussian surrogate outputs, and (ii) an analytical null derived from the known distribution of Kendall–Tau correlations. This section justifies these approximations in three steps: (i) characterizing the true null with independently initialized models, (ii) introducing the Gaussian surrogate as a tractable simulation proxy, and (iii) validating the analytical approximation.

B.1 EMPIRICAL TRUE NULL FROM INDEPENDENTLY INITIALIZED MODELS

The ideal null distribution is the distribution of correlation statistics between two independent models evaluated on random inputs. We instantiate this independence by comparing independently initialized models with different random seeds and no training. The empirical null is estimated from ~ 2500 such pairs, with Kendall–Tau correlations computed on 10,000 random inputs as in Section 3. Figure 3 shows that this true null is well described empirically by a zero-centered Gaussian distribution.

B.2 GAUSSIAN SURROGATE AS A SIMULATION-BASED APPROXIMATION

Direct estimation of the null distribution from independently initialized models is computationally expensive because it requires many model instances and repeated evaluations. We therefore introduce a tractable simulation-based null that uses Gaussian surrogates as proxies for model outputs under the null hypothesis.

This surrogate is motivated by two considerations. First, Gaussian approximations to neural-network outputs are widely used in auditing analyses and are supported by empirical and theoretical studies (Carlini et al., 2022; Lee et al., 2019). Second, our pipeline depends only on relative ordering and selection operations, such as rank correlations and bottom- m selection, and does not use semantic structure in the inputs. Randomly initialized models also have independent parameter matrices, suggesting that their null outputs should not contain strong structured dependencies. Consequently,

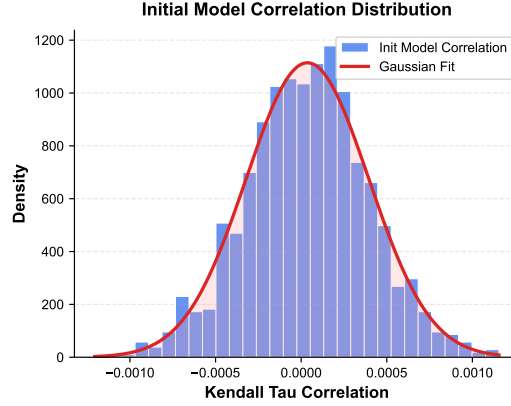


Figure 3: The empirical correlation distribution over ~ 2500 pairs of independently initialized models, evaluated on random inputs, closely matches a zero-centered Gaussian distribution.

independent Gaussian matrices provide a simple and tractable proxy for the null behavior of independently initialized models.

The surrogate null is constructed by sampling Gaussian matrices and applying the full pipeline, including identity-dimension selection and correlation computation:

1. Sample two Gaussian matrices of shape $\mathbb{R}^{N \times d_{\text{out}}}$,
2. Extract the most biased dimensions by selecting the bottom- m ranked average dimensions,
3. Take the intersection set $\mathcal{S}$ and compute correlations on it.

This simulation-based construction applies the same selection procedure to Gaussian samples and therefore captures dependency patterns introduced by the pipeline itself, including those arising from selection of $\mathcal{S}$ across dimensions. As shown in Figure 4 (middle), the resulting distribution matches the empirical null from independently initialized models in its centered Gaussian shape, indicating that the Gaussian surrogate is an effective simulation-based approximation to the true null.

B.3 ANALYTICAL APPROXIMATION UNDER WEAK DEPENDENCE

Although the Gaussian surrogate captures the full-pipeline null behavior, it still requires repeated sampling and introduces Monte Carlo randomness. We therefore also use an analytical approximation.

Under the null hypothesis that the two models are independent², each Kendall–Tau correlation τ_j is approximately zero-mean with known variance σ^2 . If correlations across identity dimensions are treated as independent, the central limit theorem implies

$$\bar{\tau} = \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} \tau_j \sim \mathcal{N}\left(0, \frac{\sigma^2}{|\mathcal{S}|}\right).$$

In practice, softmax normalization introduces dependencies across dimensions. However, Figure 4 shows that the analytical Gaussian closely matches the distribution obtained from the Gaussian surrogate pipeline across settings with varying numbers of random inputs n and output dimensions D . This agreement indicates that the dependencies are sufficiently weak and do not materially affect approximation accuracy. A plausible explanation is that the relatively high temperature ($T = 10$) smooths the output distribution and reduces inter-dimensional coupling.

Together, these results justify the analytical Gaussian null in Section 4 as an efficient, deterministic, and accurate approximation that avoids costly empirical baseline construction.

²As discussed in Section B.2, although outputs may exhibit weak dependencies due to shared inputs, this approximation remains accurate empirically.

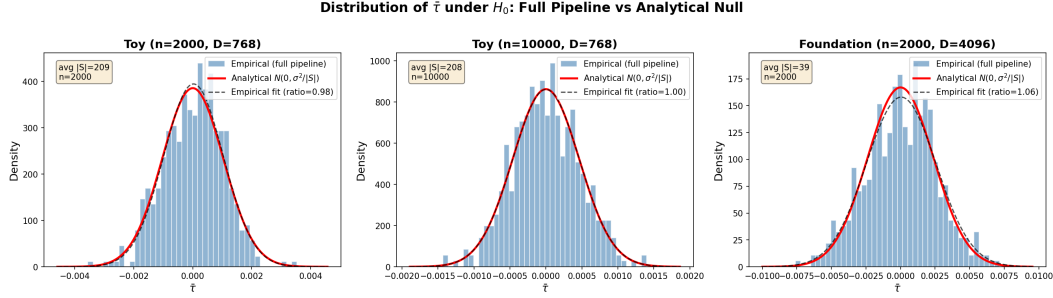


Figure 4: Validation of the analytical Gaussian null. We apply the full pipeline, including identity-dimension selection and correlation aggregation, to Gaussian surrogate outputs and compare the resulting distribution of $\bar{\tau}$ with the analytical Gaussian $\mathcal{N}(0, \sigma^2/|S|)$. The close match indicates that the analytical approximation captures null behavior accurately despite weak dependencies introduced by the pipeline.

On variance scaling. The true null from randomly initialized models has slightly smaller variance than the Gaussian null approximation. This mismatch does not affect the hypothesis-test outcomes. The primary simulation-null evaluation in Section D.2 uses the Mann–Whitney U-test, which is rank-based and invariant to positive scaling: for any $c > 0$, $x_i < x_j \Leftrightarrow cx_i < cx_j$. Thus, rank statistics and rejection thresholds are unchanged. For tests that depend on absolute variance, we also evaluate variance-sensitive statistics such as the t -test, whose statistic scales as $t' = t/c$ when variance decreases. Since $c < 1$ in our setting, scaling reduces the magnitude of t and makes the test more conservative rather than inflating significance. Across the experiments in Section D.2, variance scaling does not reduce the power of the t -test, indicating that the separation between the alternative and null distributions is substantially larger than the scaled variance.

For the analytical null used in the main paper, the variance appears explicitly in the z -score normalization, so any scale discrepancy is absorbed by the denominator. Because the analytical variance is slightly larger than the empirical variance, the resulting z -scores are smaller, making the test conservative and reducing false positives.

C THEORETICAL PROPERTIES OF OUR METHODS

The goal is to identify output dimensions that the initialized model is least likely to activate or predict under random inputs. These dimensions correspond to directions disfavored by initialization-induced bias, rather than by the semantic input distribution, and form the basis of a stable fingerprint signal.

Setup. Consider a model g with fixed parameters and random inputs x_i , instantiated as uniform random tokens or random embeddings as in the main paper. For each output dimension $j \in [d_{\text{out}}]$, $g_j(x)$ is a random variable with population mean $\bar{g}_j := \mathbb{E}[g_j(x)]$. The population-level directions of interest are the m output dimensions with the smallest expected values, formally defined as

$$\mathcal{M} = \arg \min_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} \bar{g}_j.$$

Let $(\bar{g}_{(1)}, \dots, \bar{g}_{(d_{\text{out}})})$ denote the sorted means in non-decreasing order, and write

$$\Gamma(m) := \bar{g}_{(m+1)} - \bar{g}_{(m)} > 0$$

for the population “gap” between the m -th and $(m+1)$ -th smallest means. This gap measures how well-separated the target set $\mathcal{M}$ is from the remaining coordinates. Importantly, for a fixed m and input distribution, $\Gamma(m)$ depends solely on the model’s inherent initialization bias. *However, increasing the length of random input sequences amplifies this bias via concentration, making the separation more pronounced at the population level.*

There are two natural empirical strategies to extract such population-level least-likely dimensions:

1. **Sample-mean-based extraction (ours).** Estimate the expectation of each output coordinate by the empirical mean over random inputs $X = \{x_i\}_{i=1}^n$ and select the m coordinates with the smallest

value. Following the notation in Section 4, where g denotes the model (either the base model f or the suspect model f'):

$$\hat{g} = \frac{1}{n} \sum_{i=1}^n g(x_i), \quad \widehat{\mathcal{M}}^{\text{mean}} = \arg \min_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} \hat{g}_j.$$

2. Per-sample argmin voting. For each random input $x_i \in \mathbb{R}^{\ell \times d}$, select the smallest-value coordinate and then take the m most frequent dimensions:

$$j^*(x_i) = \arg \min_{j \in \{1, \dots, d_{\text{out}}\}} g_j(x_i), \quad \widehat{\mathcal{M}}^{\text{vote}} = \arg \max_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{i=1}^n \mathbf{1}\{j^*(x_i) \in J\}.$$

Let $p_j := \Pr(j^*(x) = j)$ denote the population probability that index j attains the minimum, and let

$$\mathcal{M}_{\text{vote}} := \arg \max_{J \subseteq \{1, \dots, d_{\text{out}}\}, |J|=m} \sum_{j \in J} p_j$$

be the population top- m set under voting.

Both approaches are natural estimators of low-response dimensions. However, the analysis below shows that the sample-mean-based strategy is stable under perturbations, whereas per-sample argmin voting applies a discontinuous selection operator and can change under arbitrarily small perturbations, making it intrinsically noise-sensitive.

C.1 STABILITY ANALYSIS OF MEAN-BASED IDENTITY EXTRACTION VS. PER-SAMPLE ARGMIN VOTING

C.1.1 STABILITY OF SAMPLE-MEAN-BASED EXTRACTION

We first analyze the sample-mean-based operator, which selects the m smallest coordinates of the empirical mean vector, and show that it is stable under small perturbations to model outputs. These perturbations represent realistic output variation from the *same* model, such as numerical noise or evaluation under different precision modes.

Formally, we model perturbed outputs as $g'(x) = g(x) + \eta(x)$, where $\eta(x) \in \mathbb{R}^{d_{\text{out}}}$ represents stochastic deviations. We assume that for each sample x_i and coordinate j , the random variables $\eta_j(x_i)$ are independent across samples (not necessarily across coordinates) and σ -sub-Gaussian with $\mathbb{E}[\eta_j(x_i)] = 0$.³

Lemma C.1 (Stability of sample-mean-based extraction). *Let $\hat{g}$ and $\hat{g}'$ be the empirical means over $X = \{x_i\}_{i=1}^n$, i.e., $\hat{g} = \frac{1}{n} \sum_{i=1}^n g(x_i)$ and $\hat{g}' = \frac{1}{n} \sum_{i=1}^n g'(x_i) = \hat{g} + \frac{1}{n} \sum_{i=1}^n \eta(x_i)$. Let $\widehat{\Gamma}(m)$ denote the empirical gap at rank m . Then,*

$$\Pr(\widehat{\mathcal{M}}^{\text{mean}}(g') \neq \widehat{\mathcal{M}}^{\text{mean}}(g)) \leq 2d_{\text{out}} \exp\left(-\frac{n\widehat{\Gamma}(m)^2}{8\sigma^2}\right).$$

Interpretation. The lemma provides a quantitative upper bound on the probability that the sample-mean-based extractor changes its selected set under perturbations. This instability probability decreases exponentially with the number of averaged samples n , because averaging reduces the perturbation variance from σ^2 per sample to σ^2/n in the empirical mean.

Proof. We condition on the sample $X = \{x_i\}_{i=1}^n$ throughout the proof, so that $\hat{g}$ and $\widehat{\Gamma}(m)$ are fixed, and all probabilities are taken over the randomness of $\{\eta(x_i)\}_{i=1}^n$ only.

For any coordinate j , by definition of $\hat{g}'$, $\hat{g}'_j - \hat{g}_j = \frac{1}{n} \sum_{i=1}^n \eta_j(x_i)$. Each $\eta_j(x_i)$ is mean-zero and σ -sub-Gaussian, so their average is mean-zero and $(\sigma/\sqrt{n})$ -sub-Gaussian: the variance proxy scales

³This sub-Gaussian assumption is mild: bounded random variables are σ -sub-Gaussian (Hoeffding's lemma), and common initialization schemes together with bounded activations yield light-tailed behavior in both initialized and trained networks. The Gaussian case $\eta_j(x_i) \sim \mathcal{N}(0, \sigma^2)$ is a special case of this assumption.

as $1/n$ under averaging. Hence, for any $\varepsilon > 0$, by the standard sub-Gaussian tail bound,

$$\Pr(|\hat{g}'_j - \hat{g}_j| \geq \varepsilon \mid X) \leq 2 \exp\left(-\frac{n\varepsilon^2}{2\sigma^2}\right). \quad (1)$$

Now consider the event that *any* coordinate fluctuates by more than ε :

$$\left\{ \max_{1 \leq j \leq d_{\text{out}}} |\hat{g}'_j - \hat{g}_j| \geq \varepsilon \right\}.$$

Taking a union bound over all d_{out} coordinates yields

$$\Pr\left(\max_j |\hat{g}'_j - \hat{g}_j| \geq \varepsilon \mid X\right) \leq \sum_{j=1}^{d_{\text{out}}} \Pr(|\hat{g}'_j - \hat{g}_j| \geq \varepsilon \mid X) \leq 2d_{\text{out}} \exp\left(-\frac{n\varepsilon^2}{2\sigma^2}\right). \quad (2)$$

We now show that

$$\left\{ \widehat{\mathcal{M}}^{\text{mean}}(g') \neq \widehat{\mathcal{M}}^{\text{mean}}(g) \right\} \subseteq \left\{ \max_j |\hat{g}'_j - \hat{g}_j| \geq \widehat{\Gamma}(m)/2 \right\}.$$

Equivalently, if $\max_j |\hat{g}'_j - \hat{g}_j| < \widehat{\Gamma}(m)/2$, then the bottom- m identity set remains unchanged.

Let $j_{(1)}, \dots, j_{(d_{\text{out}})}$ be a permutation of $\{1, \dots, d_{\text{out}}\}$ such that $\hat{g}_{j_{(1)}} \leq \hat{g}_{j_{(2)}} \leq \dots \leq \hat{g}_{j_{(d_{\text{out}})}}$. Recall that the empirical gap at rank m is $\widehat{\Gamma}(m) := \hat{g}_{j_{(m+1)}} - \hat{g}_{j_{(m)}} > 0$. Denote $\widehat{\mathcal{M}}^{\text{mean}}(g) := \{j_{(1)}, \dots, j_{(m)}\}$ and $\widehat{\mathcal{M}}_c^{\text{mean}}(g) := \{j_{(m+1)}, \dots, j_{(d_{\text{out}})}\}$. For any $i \in \widehat{\mathcal{M}}^{\text{mean}}(g)$ and any $\ell \in \widehat{\mathcal{M}}_c^{\text{mean}}(g)$, $\hat{g}_\ell - \hat{g}_i \geq \hat{g}_{j_{(m+1)}} - \hat{g}_{j_{(m)}} = \widehat{\Gamma}(m)$. With $|\hat{g}'_i - \hat{g}_i| < \widehat{\Gamma}(m)/2$ and $|\hat{g}'_\ell - \hat{g}_\ell| < \widehat{\Gamma}(m)/2$, we have

$$\hat{g}'_\ell - \hat{g}'_i \geq (\hat{g}_\ell - \frac{\widehat{\Gamma}(m)}{2}) - (\hat{g}_i + \frac{\widehat{\Gamma}(m)}{2}) \geq \widehat{\Gamma}(m) - \widehat{\Gamma}(m) = 0.$$

Thus no coordinate in $\widehat{\mathcal{M}}_c^{\text{mean}}(g)$ can become smaller than any coordinate in $\widehat{\mathcal{M}}^{\text{mean}}(g)$ under this deviation bound, and therefore

$$\widehat{\mathcal{M}}^{\text{mean}}(g') = \widehat{\mathcal{M}}^{\text{mean}}(g).$$

Since $\widehat{\Gamma}(m)$ is fixed under conditioning on X , plugging $\varepsilon = \widehat{\Gamma}(m)/2$ into (2) yields

$$\Pr\left(\widehat{\mathcal{M}}^{\text{mean}}(g') \neq \widehat{\mathcal{M}}^{\text{mean}}(g)\right) \leq \Pr\left(\max_j |\hat{g}'_j - \hat{g}_j| \geq \widehat{\Gamma}(m)/2 \mid X\right) \leq 2d_{\text{out}} \exp\left(-\frac{n\widehat{\Gamma}(m)^2}{8\sigma^2}\right).$$

□

C.1.2 INSTABILITY OF PER-SAMPLE ARGMIN VOTING

In contrast, per-sample voting applies a discontinuous winner-take-all operator to each input. Consequently, arbitrarily small output perturbations can change the selected coordinate with non-vanishing probability. We use the same stochastic perturbation model $g'(x) = g(x) + \eta(x)$.

Lemma C.2 (Instability of per-sample argmin voting). *For a fixed input x , let*

$$j^*(x; g) := \arg \min_j g_j(x), \quad \Delta_g(x) := \min_{j \neq j^*(x; g)} \{g_j(x) - g_{j^*(x; g)}(x)\} > 0$$

denote the argmin and its margin. Then, for any fixed x ,

$$\Pr(j^*(x; g') \neq j^*(x; g) \mid x) \geq \Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right),$$

and for n i.i.d. inputs $x_1, \dots, x_n$,

$$\Pr\left(\exists i \leq n : j^*(x_i; g') \neq j^*(x_i; g)\right) \geq 1 - (1 - \alpha)^n,$$

where $\alpha := \mathbb{E}_x \left[\Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right) \right]$.

Interpretation. Unlike mean extraction, voting does not obtain variance reduction before the hard decision is made: each input produces a direct discrete choice from noisy outputs. Increasing n therefore also increases the number of opportunities for index flips.

Proof. We first bound the per-input change probability $\Pr(j^*(x; g') \neq j^*(x; g) \mid x)$. Fix x and write $j^* := j^*(x; g)$ for brevity. By definition of the margin, $\Delta_g(x) = \min_{j \neq j^*} \{g_j(x) - g_{j^*}(x)\}$. Let $j^{(2)}$ be an index attaining this minimum, i.e., $g_{j^{(2)}}(x) - g_{j^*}(x) = \Delta_g(x)$.

Under the perturbation $g'(x) = g(x) + \eta(x)$, we have

$$g'_{j^{(2)}}(x) - g'_{j^*}(x) = [g_{j^{(2)}}(x) - g_{j^*}(x)] + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)] = \Delta_g(x) + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)].$$

Since $\eta_{j^{(2)}}(x)$ and $\eta_{j^*}(x)$ are independent $\mathcal{N}(0, \sigma^2)$ variables, their difference is Gaussian with

$$\eta_{j^{(2)}}(x) - \eta_{j^*}(x) \sim \mathcal{N}(0, 2\sigma^2).$$

The event that the argmin changes, $j^*(x; g') \neq j^*(x; g)$, certainly occurs whenever $g'_{j^{(2)}}(x) \leq g'_{j^*}(x)$. Hence,

$$\Pr(j^*(x; g') \neq j^*(x; g) \mid x) \geq \Pr(g'_{j^{(2)}}(x) \leq g'_{j^*}(x) \mid x).$$

Using the expression above,

$$\Pr(g'_{j^{(2)}}(x) \leq g'_{j^*}(x) \mid x) = \Pr(g'_{j^{(2)}}(x) - g'_{j^*}(x) \leq 0 \mid x) = \Pr(\Delta_g(x) + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)] \leq 0 \mid x).$$

Let $Z \sim \mathcal{N}(0, 1)$. Since $\eta_{j^{(2)}}(x) - \eta_{j^*}(x) \stackrel{d}{=} \sqrt{2}\sigma Z$, we obtain

$$\Pr(\Delta_g(x) + [\eta_{j^{(2)}}(x) - \eta_{j^*}(x)] \leq 0 \mid x) = \Pr\left(Z \leq -\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right) = \Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right).$$

Therefore, $\Pr(j^*(x; g') \neq j^*(x; g) \mid x) \geq \Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right)$.

Now consider n i.i.d. inputs $x_1, \dots, x_n$. Conditional on the inputs, the events

$$E_i := \{j^*(x_i; g') \neq j^*(x_i; g)\} \quad E_i^c := \{j^*(x_i; g') = j^*(x_i; g)\}$$

are independent across i , because the perturbations $\eta(x_i)$ are independent across i . Moreover,

$$\Pr(E_i \mid x_i) = \Pr(j^*(x_i; g') \neq j^*(x_i; g) \mid x) \geq \Phi\left(-\frac{\Delta_g(x_i)}{\sqrt{2}\sigma}\right) \geq \Phi\left(-\frac{\Delta_g(x_i)}{\sqrt{2}\sigma}\right).$$

Define $q(x_i) := \Pr(j^*(x_i; g') \neq j^*(x_i; g) \mid x)$, and $\alpha := \mathbb{E}_x\left[\Phi\left(-\frac{\Delta_g(x)}{\sqrt{2}\sigma}\right)\right]$. Taking expectations over the randomness of x_i , we obtain $\mathbb{E}[\Pr(E_i \mid x_i)] = \alpha$. Therefore, by independence across i ,

$$\Pr(E_1^c \cap \dots \cap E_n^c) = \mathbb{E}\left[\prod_{i=1}^n (1 - q(x_i))\right] \leq \mathbb{E}\left[\prod_{i=1}^n (1 - \Phi(-\Delta_g(x_i)/(\sqrt{2}\sigma)))\right] \leq (1 - \alpha)^n,$$

where the last inequality uses i.i.d. inputs and Jensen's inequality applied to the convex function $u \mapsto \log(1 - u)$ on $u \in [0, 1)$. Therefore,

$$\Pr(\exists i \leq n : j^*(x_i; g') \neq j^*(x_i; g)) = 1 - \Pr(E_1^c \cap \dots \cap E_n^c) \geq 1 - (1 - \alpha)^n.$$

Finally, the empirical voting set $\widehat{\mathcal{M}}^{\text{vote}}$ is determined by the empirical counts of the dimensions $\{j^*(x_i; g)\}_{i=1}^n$. If the empirical gap between the m -th and $(m+1)$ -th most frequent dimensions is $O(1/n)$, i.e., only a constant number of votes, then changing $j^*(x_i)$ for a single input x_i can swap the ordering across this boundary and alter the top- m set. Combining this observation with the lower bound $1 - (1 - \alpha)^n$ shows that, in such regimes, the probability that $\widehat{\mathcal{M}}^{\text{vote}}(g') \neq \widehat{\mathcal{M}}^{\text{vote}}(g)$ is bounded away from zero uniformly in n , i.e., per-sample argmin voting does not enjoy variance reduction with more samples. $\square$

C.2 INFLUENCE OF HYPERPARAMETERS

We next analyze the number of random inputs required for the empirical mean estimator to recover the true bias set. We reuse notation from **Setup**: each output coordinate $g_j(x)$ is a random variable under random inputs $x \sim \mathcal{D}$, with population mean $\bar{g}_j := \mathbb{E}[g_j(x)]$. $(\bar{g}_{(1)}, \dots, \bar{g}_{(d_{\text{out}})})$ represents the sorted means in non-decreasing order, and $\gamma := \bar{g}_{(m+1)} - \bar{g}_{(m)} > 0$ is the separation margin. We estimate $\bar{g}$ using n i.i.d. random inputs:

$$\hat{g}_j := \frac{1}{n} \sum_{i=1}^n g_j(x_i), \quad \hat{g} = (\hat{g}_1, \dots, \hat{g}_{d_{\text{out}}}).$$

The target dimension set is $\mathcal{M}$ with cardinality m , and the empirical estimator produces $\widehat{\mathcal{M}}$.

Lemma C.3 (Sufficient condition for recovering the true set). *If $\|\hat{g} - \bar{g}\|_\infty \leq \varepsilon$ and $2\varepsilon < \gamma$, then the empirical bottom- m set equals the true set: $\widehat{\mathcal{M}} = \mathcal{M}$.*

Proof. For any $j \in \mathcal{M}$ and $j' \notin \mathcal{M}$, we have $\bar{g}_j \leq \bar{g}_{(m)}$ and $\bar{g}_{j'} \geq \bar{g}_{(m+1)}$. The ℓ_∞ bound gives

$$\hat{g}_j \leq \bar{g}_{(m)} + \varepsilon, \quad \hat{g}_{j'} \geq \bar{g}_{(m+1)} - \varepsilon.$$

Thus,

$$\hat{g}_j - \hat{g}_{j'} \leq -(\gamma - 2\varepsilon) < 0,$$

so no outside index can enter nor inside index leave the bottom- m set. $\square$

We next express this recovery condition in terms of hyperparameters. We use the same sub-Gaussian assumption as in Section C.1. This assumption is mild: by Hoeffding's inequality, any bounded random variable is σ -sub-Gaussian with σ proportional to its range. In this setting, each $g_j(x)$ is a logit or final hidden activation, which is typically bounded or light-tailed in both initialized networks, due to controlled parameter initialization, and trained networks.

Corollary C.4 (Sample complexity under sub-Gaussian coordinates). *Suppose each $g_j(x)$ is σ -sub-Gaussian. Then with probability at least $1 - \delta$,*

$$\|\hat{g} - \bar{g}\|_\infty \leq \sigma \sqrt{\frac{2 \log(2d_{\text{out}}/\delta)}{n}}.$$

Consequently, if

$$n \geq \frac{8\sigma^2}{\gamma^2} \log\left(\frac{2d_{\text{out}}}{\delta}\right),$$

then $\widehat{\mathcal{M}} = \mathcal{M}$ with probability at least $1 - \delta$.

Proof. For fixed j , sub-Gaussian concentration implies

$$\Pr(|\hat{g}_j - \bar{g}_j| > \varepsilon) \leq 2 \exp(-n\varepsilon^2/2\sigma^2).$$

A union bound over all d_{out} coordinates yields

$$\Pr\left(\|\hat{g} - \bar{g}\|_\infty > \varepsilon\right) \leq \sum_{j=1}^{d_{\text{out}}} \Pr(|\hat{g}_j - \bar{g}_j| > \varepsilon) \leq 2d_{\text{out}} \exp\left(-\frac{n\varepsilon^2}{2\sigma^2}\right).$$

Set the right-hand side to δ and solve for ε to obtain, with probability at least $1 - \delta$,

$$\|\hat{g} - \bar{g}\|_\infty \leq \sigma \sqrt{\frac{2 \log(2d_{\text{out}}/\delta)}{n}}.$$

Now choose

$$\varepsilon := \sigma \sqrt{\frac{2 \log(2d_{\text{out}}/\delta)}{n}}.$$

If $n \geq \frac{8\sigma^2}{\gamma^2} \log\left(\frac{2d_{\text{out}}}{\delta}\right)$ (i.e., if $2\varepsilon < \gamma$), then Lemma C.3 applies and implies $\widehat{\mathcal{M}} = \mathcal{M}$ with probability at least $1 - \delta$. $\square$

Table 9: Fingerprint behavior induced by different random initializations across model families.

(a) LLaMA-style models					(b) Qwen-style models				
Seed Pair	Logits Output		Hidden State		Seed Pair	Logits Output		Hidden State	
	<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test		<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test
s_{42} vs. s_{2000}	0.404	0.456	0.357	0.532	s_{123} vs. s_{1000}	0.741	0.727	0.094	0.074
s_{123} vs. s_{42}	0.214	0.295	0.678	0.565	s_{1000} vs. s_{123}	0.954	0.971	0.125	0.094
s_{1000} vs. s_{123}	0.219	0.246	0.363	0.335	s_{42} vs. s_{2000}	0.273	0.360	0.451	0.529
s_{2000} vs. s_{1000}	0.282	0.291	0.434	0.481	s_{2000} vs. s_{42}	0.215	0.206	0.230	0.295

Table 10: (LLaMA-style models) Trained models preserve the fingerprint behavior of their initialization (p -value < 0.01).

Model Pair	Logits Output		Hidden State		Baselines			
	<i>t</i> -test	<i>u</i> -test	<i>t</i> -test	<i>u</i> -test	Intrinsic	REEF	PCS	ICS
s_{42}^{init} vs. s_{42}^{base}	3.33e-3✓	1.02e-3✓	2.20e-8✓	6.28e-8✓	-0.021×	0.375×	0.580×	0.196×
s_{123}^{init} vs. s_{123}^{base}	2.06e-3✓	7.33e-3✓	7.09e-6✓	1.37e-5✓	0.149×	0.369×	0.581×	0.188×
s_{1000}^{init} vs. s_{1000}^{base}	2.44e-3✓	4.14e-3✓	5.58e-4✓	2.81e-3✓	-0.252×	0.381×	0.581×	0.188×
s_{2000}^{init} vs. s_{2000}^{base}	5.63e-3✓	6.76e-3✓	4.00e-10✓	1.27e-9✓	-0.337×	0.331×	0.581×	0.188×

Therefore, accurate recovery requires $n = \Omega\left(\frac{\log d_{out}}{\gamma^2}\right)$: the number of random queries grows logarithmically with output dimension and inversely with the squared separation margin γ . The margin γ is affected by two factors:

- **Intrinsic bias strength.** Longer random input sequences amplify initialization-induced bias via concentration, increasing the population separation between coordinates.
- **Choice of m .** Different choices of m correspond to different parts of the ranked spectrum $(\bar{g}_{(1)}, \dots, \bar{g}_{(d_{out})})$, thus yielding different effective margins in practice.

The former is controlled by the input length l , whereas the latter is selected empirically from the observed gaps in the mean spectrum.

D MORE EXPERIMENT DETAILS

D.1 IMPLEMENTATION DETAILS AND LICENSES

All models are trained with the Hugging Face Transformers Trainer (Wolf et al., 2020), with Accelerate (Gugger et al., 2022) used for distributed execution. Open-source models are loaded from their official Hugging Face releases and used under their original licenses: Llama models under the Meta Llama Community License and the remaining models under Apache-2.0. Datasets are downloaded through the Hugging Face Datasets library, which is Apache-2.0 licensed; dataset content follows each dataset’s stated license.

D.2 COMPLEMENTARY SEEDPRINTS RESULTS USING SIMULATION-BASED NULL

This section reports complementary results for Section 5. We use the empirical simulation-based null from Algorithm 1 with both *logits* and *hidden state* outputs, and evaluate both the one-sided *t*-test (*t*-test) and the Mann–Whitney *U* test (*U* test).

D.2.1 BIRTH-TO-LIFECYCLE “BIOMETRIC” FINGERPRINTING

We train 12-layer, 12-head LLaMA-style models (Touvron et al., 2023) with RoPE (Su et al., 2021), as well as Qwen-style models (Team, 2024), from scratch. Because the simulation-based random baseline is stochastic, we report p -values averaged over 10 independent trials and use significance level $\alpha = 0.01$. The absolute magnitude of extremely small p -values is not meaningful: once p is

Table 11: (Qwen-style models) Trained models preserve the fingerprint behavior of their initialization models (p -value < 0.01).

Model Pair	Logits Output		Hidden State	
	t -test	U -test	t -test	U -test
s_{123}^{init} vs. s_{123}^{base}	2.38e-03	1.68e-03	7.36e-15	3.38e-13
s_{1000}^{init} vs. s_{1000}^{base}	1.35e-04	4.84e-05	4.41e-13	2.01e-11
s_{42}^{init} vs. s_{42}^{base}	1.39e-03	1.46e-03	1.06e-24	2.05e-19
s_{2000}^{init} vs. s_{2000}^{base}	1.80e-03	1.28e-03	4.87e-24	1.92e-20

Table 12: Fingerprint persistence under continual training on diverse datasets, using the seed-1000 base model pretrained on openwebtext. U -test denotes the Mann–Whitney U test.

(a) LLaMA-style models

Setting	Ours (logits)		Ours (hidden)		Baselines			
Continual corpus (seed)	t -test	U -test	t -test	U -test	Intrinsic	REEF	PCS	ICS
TinyStories (1000)	0✓	0✓	0✓	7.77e-89✓	1.000✓	0.759×	0.999✓	0.996✓
TinyStories (123)	1.000✓	1.00✓	0.943✓	0.902✓	0.950×	0.658✓	0.332✓	0.012✓
the_stack (1000)	0✓	1.73e-287✓	0✓	3.09e-69✓	0.489×	0.557×	0.585×	0.123×
the_stack (123)	0.616✓	0.479✓	0.732✓	0.831✓	0.445✓	0.580✓	0.301✓	0.026✓

(b) Qwen-style models

Setting	Ours (logits)		Ours (hidden)		Baselines			
Continual corpus (seed)	t -test	U -test	t -test	U -test	Intrinsic	REEF	PCS	ICS
TinyStories (1000)	5.36e-09✓	1.92e-07✓	8.49e-214✓	5.09e-71✓	1.000✓	0.957✓	0.999✓	0.996✓
TinyStories (123)	0.434✓	0.433✓	0.256✓	0.065✓	0.913×	0.199✓	0.328✓	0.039✓
the_stack (1000)	0✓	4.16e-237✓	1.16e-211✓	2.30e-76✓	0.999✓	0.313×	0.995✓	0.976✓
the_stack (123)	0.999✓	0.993✓	0.610✓	0.491✓	0.916×	0.255✓	0.328✓	0.038✓

below numerical and sampling noise (e.g., $< 10^{-20}$), values such as 10^{-260} should not be interpreted as stronger evidence than 10^{-20} ; both decisively reject the null.

Different initialization seeds produce distinct fingerprints Table 9a and Table 9b report p -values from correlation tests between model pairs initialized with different random seeds (42, 123, 1000, and 2000), using the t -test and U -test for Llama-style and Qwen-style models, respectively. All p -values remain > 0.01 , indicating that the method distinguishes models trained from different seeds. Thus, distinct seeds induce distinct fingerprint behavior, enabling separation at model “birth.”

Training preserves the initialization fingerprint. Table 10 (LLaMA-style) and Table 11 (Qwen-style) compare each initialization model s^{init} with its descendant s^{base} trained on OpenWebText (Gokaslan et al., 2019) (≈ 10 B tokens). Across all seed–model pairs, p -values are consistently < 0.01 , showing that the bias profiles remain strongly correlated and therefore share lineage. The trained model therefore inherits the initialization fingerprint; training does not erase it. We also evaluate baseline methods in Table 10; all baselines fail to distinguish seeds, suggesting that their separability derives from training-induced artifacts rather than initialization.

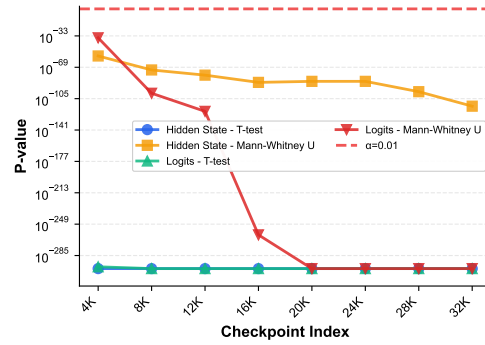
Identical data and order do not make fingerprints converge In Table 13a (LLaMA-style models) and Table 13b (Qwen-style models), all four “suspicious” models s_i^{base} for $i \in \{42, 123, 100, 2000\}$ are trained on *exactly the same corpus (OpenWebText) and in the same data order*; the training seed is fixed to control the data order. The only varying factor is the initialization seed i . This experiment tests whether identical data exposure and order erase or confound fingerprint behavior. Across all cross-seed pairs, p -values remain consistently > 0.01 , in contrast to the near-zero values in Table 10. Fingerprints therefore remain seed-specific even under identical data and curriculum.

Table 13: Identical datasets and training order do not make fingerprint behavior identical across different initializations.

(a) LLaMA-style models					(b) Qwen-style models				
Model Pair	Logits Output		Hidden State		Model Pair	Logits Output		Hidden State	
	<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test		<i>t</i> -test	<i>U</i> -test	<i>t</i> -test	<i>U</i> -test
s_{123}^{init} vs. s_{1000}^{base}	0.484	0.500	0.385	0.486	s_{123}^{init} vs. s_{1000}^{base}	0.598	0.638	0.286	0.254
s_{1000}^{init} vs. s_{2000}^{base}	0.946	0.956	0.135	0.096	s_{1000}^{init} vs. s_{123}^{base}	0.804	0.805	0.236	0.240
s_{42}^{init} vs. s_{123}^{base}	0.598	0.589	0.426	0.337	s_{42}^{init} vs. s_{2000}^{base}	0.589	0.608	0.226	0.2043
s_{2000}^{init} vs. s_{42}^{base}	0.756	0.781	0.388	0.287	s_{2000}^{init} vs. s_{42}^{base}	0.523	0.482	0.312	0.323



(a) LLaMA-style models



(b) Qwen-style models

Figure 5: The fingerprint verifies lineage at every checkpoint (p -values < 0.01) across model families.

Continual training on diverse datasets does not confound the fingerprint The preceding experiments are intended to demonstrate that SeedPrints can function as a biometric fingerprint. From a copyright perspective, the inability of prior work to distinguish different initialization seeds is not inherently problematic, because initialization seeds do not have clear copyright status. The more important failure mode is that prior attribution signals can be driven by *data distribution*, thereby *failing to recognize lineage when the training distribution shifts substantially during continued training*.

In Table 12a and Table 12b, we continue training a seed-1000 base model pretrained on OpenWebText (Gokaslan et al., 2019) on two highly different datasets: TinyStories (Eldan & Li, 2023), consisting of synthetic children’s stories, and The Stack (Kocetkov et al., 2022), consisting of permissively licensed GitHub code. We compare (i) true descendants trained from the base with (ii) *distractors* derived from a different base model initialized with seed 123, trained on OpenWebText with a different data order, and then continued on the *same* target corpus. The evaluation tests whether attribution methods identify the descendant that actually shares lineage with the base.

Prior baselines all fail in the code-domain setting (The Stack), misclassifying true descendants as distractors. This indicates that their scores largely track domain similarity rather than lineage identity: TinyStories is closer to the OpenWebText pretraining distribution, whereas The Stack is substantially different, and this distribution shift is sufficient to bypass detection. SeedPrints correctly attributes lineage across both corpora. Thus, the fingerprint is not a proxy for data distribution; it persists under substantial domain shift and beyond the initial pretraining stage.

D.2.2 ALL-STAGE VERIFIABLE FINGERPRINTS

Controlled pretraining trajectory. We first conduct a controlled OpenWebText pretraining experiment in which the initialization seed is known. Each intermediate checkpoint is treated as the base, and the test evaluates whether the method identifies its descendant along the same training trajectory. All variants consistently classify the suspect model as same-lineage, with p -values below the 0.01 threshold (Figure 5a for LLaMA-style models and Figure 5b for Qwen-style models). This result confirms the stability of SeedPrints under continuous optimization.

Table 14: Cross-size fingerprint comparison between Qwen-2.5-14B (target) and other Qwen-2.5 models.

	Qwen-2.5-0.5B	Qwen-2.5-1.5B	Qwen-2.5-3B	Qwen-2.5-7B
SeedPrint ($\uparrow$)	0.7184	0.9707	0.3898	0.5429
REEF ($\downarrow$)	0.8250	0.8399	0.8833	0.8461
HUREF ($\downarrow$)	-6.37×10^{-5}	-2.84×10^{-5}	-8.70×10^{-6}	-6.08×10^{-6}

D.3 CROSS-SIZE FINGERPRINT WITHIN THE QWEN-2.5 SERIES

To examine whether SeedPrints collapses to a family-level signature when parameter size varies, we report cross-size comparisons between Qwen-2.5-14B as the target and smaller models from the same Qwen-2.5 family. Table 14 reports SeedPrint ($\uparrow$) and two baselines, REEF ($\downarrow$) and HUREF ($\downarrow$).

We conduct the cross-size study within the Qwen-2.5 series. Qwen-2.5-14B is treated as the protected target model, and Qwen-2.5-{0.5B, 1.5B, 3B, 7B} are treated as suspect models that share the architectural family but are not descendants of Qwen-2.5-14B. As discussed in the main paper, SeedPrints outputs a p -value for the hypothesis test “the suspect shares initialization-born fingerprints with the target”, whereas REEF and HUREF output continuous similarity scores and require a user-specified threshold.

The results support three observations. First, all SeedPrints p -values are well above a typical significance level such as $\alpha=0.01$. Under the hypothesis-testing interpretation, none of the smaller Qwen-2.5 models is flagged as sharing lineage with Qwen-2.5-14B. This behavior is desirable: although the models belong to the same family and reuse a similar architectural design, the test does not collapse to a family-level signature that spuriously conflates distinct pretraining runs.

Second, the cross-size p -values occupy different regions of the $[0, 1]$ interval rather than concentrating at a single value. We do *not* interpret these magnitudes as continuous “similarity scores”; their role is to support a calibrated binary decision at a fixed significance level. Nevertheless, the spread indicates that the test statistic remains sensitive to the specific combination of architecture and initialization seed, rather than degenerating into a generic pattern shared by all Qwen-2.5 variants.

Third, the similarity-based baselines exhibit different behavior. REEF produces scores in a narrow range (0.8250–0.8833) across all sizes, whereas HUREF outputs values numerically close to zero, on the order of 10^{-5} . Because these methods do not provide an intrinsic decision rule, operators must choose thresholds manually, and a single cutoff may not robustly separate cross-size variants from genuine lineage-related models. In contrast, SeedPrints directly returns a statistical decision at a prescribed significance level, yielding an operational answer to the question “do these two models plausibly share a training lineage?” within a model family.

E MODEL CATALOG AND DECODING SETTINGS

The evaluation covers **58** models in total. This section documents the decoding configuration used in all runs and enumerates the model catalog by family. For each derived model, we also specify the post-training transformation (*finetune*, *adapter*, *merge*, *quantization*, *distillation*). “PT” and “IT” denote *pre-trained* and *instruction-tuned* source models, respectively.

Parameter	Value
max_new_tokens	512
temperature	0.7
top_p	0.9
top_k	50
do_sample	true
max_input_length	512

E.1 MODEL LIST BY FAMILY

E.1.1 QWEN-2.5-7B

Variant	HuggingFace repo	Type
PT	Qwen/Qwen2.5-7B	—
IT	Qwen/Qwen2.5-7B-Instruct	—
Derived	Qwen/Qwen2.5-Math-7B	finetune
Derived	Qwen/Qwen2.5-Coder-7B-Instruct	finetune
Derived	WangCa/Qwen2.5-7B-Medicine	finetune
Derived	huihui-ai/Qwen2.5-7B-Instruct-abliterated-v2	finetune
Derived	Locutusque/StockQwen-2.5-7B	merge
Derived	bunnycore/QevaCoT-7B-Stock	merge
Derived	fangcaotank/task-10-Qwen-Qwen2.5-7B-Instruct	adapter
Derived	SeeFlock/task-12-Qwen-Qwen2.5-7B-Instruct	adapter
Derived	Qwen/Qwen2.5-7B-Instruct-GPTQ-Int4	quantization
Derived	Qwen/Qwen2.5-7B-Instruct-GPTQ-Int8	quantization
Derived	Lansechen/Qwen2.5-7B-Open-R1-Distill	distillation

E.1.2 QWEN2.5-14B

Variant	HuggingFace repo	Type
PT	Qwen/Qwen2.5-14B	—
IT	Qwen/Qwen2.5-14B-Instruct	—
Derived	Qwen/Qwen2.5-Coder-14B	finetune
Derived	oxyapi/oxy-1-small	finetune
Derived	v000000/Qwen2.5-14B-Gutenberg-Instruct-Slerpeno	merge
Derived	ToastyPigeon/qwen-story-test-qlora	adapter
Derived	Qwen/Qwen2.5-14B-Instruct-GPTQ-Int4	quantization
Derived	deepseek-ai/DeepSeek-R1-Distill-Qwen-14B	distillation

E.1.3 LLAMA-3.1-8B

Variant	HuggingFace repo	Type
PT	meta-llama/Llama-3.1-8B	—
IT	meta-llama/Llama-3.1-8B-Instruct	—
Derived	ValiantLabs/Llama3.1-8B-Fireplace2	finetune
Derived	RedHatAI/Llama-3.1-8B-tldr	finetune
Derived	proxectonos/Llama-3.1-Carballo	finetune
Derived	mlabonne/Meta-Llama-3.1-8B-Instruct-abliterated	finetune
Derived	gaverfraxz/Meta-Llama-3.1-8B-Instruct-HalfAbliterated-TIES	merge
Derived	XiaoJian9992024/Llama3.1-8B-ExtraMix	merge
Derived	LlamaFactoryAI/Llama-3.1-8B-Instruct-cv-job-description-matching	adapter
Derived	chchen/Llama-3.1-8B-Instruct-PsyCourse-fold7	adapter
Derived	iqbalamo93/Meta-Llama-3.1-8B-Instruct-GPTQ-Q_8	quantization
Derived	DaraV/LLaMA-3.1-8B-Instruct-INT4-GPTQ	quantization
Derived	asas-ai/Llama-3.1-8B-Instruct-Open-R1-Distill	distillation

E.1.4 MISTRAL-7B-v0.3

Variant	HuggingFace repo	Type
PT	mistralai/Mistral-7B-v0.3	—
IT	mistralai/Mistral-7B-Instruct-v0.3	—
Derived	KurmaAI/AQUA-7B	finetune
Derived	openfoodfacts/spellcheck-mistral-7b	finetune
Derived	grimjim/Mistral-7B-Instruct-demi-merge-v0.3-7B	merge
Derived	chaymaemrhrioui/mistral-Brain_Model_ACC_Trainer	adapter
Derived	RedHatAI/Mistral-7B-Instruct-v0.3-GPTQ-4bit	quantization
Derived	eganwo/mistral7b-distilled-from-deepseek-r1-qwen32b	distillation

E.1.5 GEMMA-2-2B

Variant	HuggingFace repo	Type
PT	google/gemma-2-2b	—
IT	google/gemma-2-2b-it	—
Derived	rinna/gemma-2-baku-2b	finetune
Derived	anakin87/gemma-2-2b-neogenesis-ita	finetune
Derived	vonjack/gemma2-2b-merged	merge
Derived	google-cloud-partnership/gemma-2-2b-it-lora-sql	adapter
Derived	qilowoq/gemma-2-2B-it-4Bit-GPTQ	quantization
Derived	Syed-Hasan-8503/Gemma-2-2b-it-distilled	distillation

E.1.6 LLAMA-2-7B

Variant	HuggingFace repo	Type
PT	meta-llama/Llama-2-7b-hf	—
IT	meta-llama/Llama-2-7b-chat-hf	—
Derived	allenai/tulu-2-7b	finetune
Derived	QIAIUNCC/EYE-Llama_qa	finetune
Derived	DevQuasar/coma-7B-v0.1	merge
Derived	Ammar-1/llama2-Better-Tune	adapter
Derived	TheBloke/Llama-2-7B-Chat-GPTQ	quantization
Derived	cygu/llama-2-7b-logit-watermark-distill-kgw-k1-gamma0.25-delta2	distillation

E.1.7 TINYLLAMA-1.1B

Variant	HuggingFace repo	Type
PT/IT	TinyLlama/TinyLlama-1.1B-Chat-v1.0	—
Derived	alexredna/TinyLlama-1.1B-Chat-v1.0-reasoning-v2	finetune
Derived	Edentns/DataVortexTL-1.1B-v0.1	finetune
Derived	appvoid/dot-v2.7	merge
Derived	barissglc/tinyllama-tarot-v1	adapter
Derived	TheBloke/TinyLlama-1.1B-Chat-v1.0-GPTQ	quantization
Derived	anudaw/distilled-code-llama	distillation

Notes. “Type” denotes the post-training transformation relative to the PT/IT source model: *finetune* includes supervised/preference optimization variants; *adapter* includes LoRA/QLoRA-style modules; *merge* includes model soups and TIES-style merges; *quantization* includes GPTQ/INTx variants; *distillation* includes student models distilled from reasoning-augmented teachers.

E.2 SAMPLE RESULTS

Table 15 reports partial results from one LeafBench evaluation run using the u -test with the simulation-based null. The reported metric is $1 - p$, used as a similarity-style score for compatibility with the benchmark. We use a significance level of 0.01 for p -values; empirically, LeafBench identifies an optimal decision threshold of 0.9920, further supporting the reliability of the method.

Crucially, individual p -values should not be interpreted as linear similarity measures, but their distribution across model pairs is informative: same-lineage models yield substantially lower p -values, whereas unrelated models remain close to the null distribution.

1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619

Model	Qwen-2.5-7B	Qwen-2.5-7B-Instruct	Qwen2.5-14B	Qwen2.5-14B-Instruct
Qwen-2.5-7B	1.000	1.000	0.079	0.000
Qwen-2.5-7B-Instruct	1.000	1.000	0.813	0.536
Qwen2.5-7B-Math	1.000	0.994	0.047	0.987
Qwen2.5-7B-Coder	0.996	0.999	0.177	0.002
Qwen2.5-7B-Instruct-Medicine	1.000	1.000	0.576	0.369
Qwen2.5-7B-Instruct-Abiliterated	1.000	1.000	0.276	0.886
Qwen2.5-7B-Stock	1.000	1.000	0.939	0.225
QevaCoT-7B	1.000	1.000	0.199	0.119
Qwen2.5-7B-Instruct-Task-10	1.000	1.000	0.672	0.090
Qwen2.5-7B-Instruct-Task-12	1.000	1.000	0.155	0.987
Qwen2.5-7B-Instruct-Int4	1.000	1.000	0.827	0.961
Qwen2.5-7B-Instruct-Int8	1.000	1.000	0.914	0.739
Qwen2.5-7B-Open-R1-Distill	1.000	1.000	0.302	0.955
Qwen2.5-14B	0.324	0.487	1.000	1.000
Qwen2.5-14B-Instruct	0.403	0.702	1.000	1.000
Qwen2.5-Coder-14B	0.492	0.660	1.000	1.000
oxy-1-small	0.492	0.104	1.000	1.000
Qwen2.5-14B-Gutenberg-Instruct-Slerpeno	0.856	0.931	1.000	1.000
Qwen-story-test-qlora	0.112	0.898	1.000	1.000
Qwen2.5-14B-Instruct-GPTQ-Int4	0.583	0.092	1.000	1.000
DeepSeek-R1-Distill-Qwen-14B	0.204	0.000	1.000	1.000
Llama-3.1-8B	0.638	0.714	0.156	0.000
Llama-3.1-8B-Instruct	0.643	0.118	0.128	0.510
Llama-3.1-8B-Fireplace2	0.715	0.532	0.065	0.354
Llama-3.1-8B-TLDR	0.925	0.811	0.331	0.014
Llama-3.1-8B-Carballo	0.001	0.605	0.003	0.946
Llama-3.1-8B-Instruct-Abliterated	0.061	0.005	0.121	0.422
Llama-3.1-8B-Instruct-HalfAbliterated-TIES	0.356	0.509	0.796	0.000
Llama-3.1-8B-ExtraMix	0.640	0.160	0.721	0.875
Llama-3.1-8B-Instruct-cv-job-description-matching	0.684	0.292	0.004	0.001
Llama-3.1-8B-Instruct-PsyCourse-fold7	0.682	0.057	0.063	0.191
Llama-3.1-8B-Instruct-8bit	0.798	0.214	0.154	0.385
Llama-3.1-8B-Instruct-4bit	0.513	0.023	0.005	0.933
Llama-3.1-8B-Instruct-Open-R1-Distill	0.800	0.101	0.772	0.024
Mistral-7B-v0.3	0.224	0.352	0.506	0.044
Mistral-7B-v0.3-Instruct	0.000	0.000	0.984	0.104
AQUA-7B	0.014	0.613	0.703	0.569
Mistral-7B-v0.3-Spellcheck	0.005	0.481	0.872	0.105
Mistral-7B-v0.3-Instruct-demi-merge	0.904	0.774	0.058	0.522
Mistral-7B-v0.3-Brain	0.340	0.983	0.481	0.016
Mistral-7B-v0.3-Instruct-GPTQ-4bit	0.002	0.281	0.089	0.784
Mistral-7B-distilled-from-deepseek-r1-qwen32b	0.009	0.001	0.511	0.015
Gemma-2-2b	0.747	0.869	0.689	0.000
Gemma-2-2b-it	0.944	0.039	0.626	0.060
Gemma-2-baku-2b	0.616	0.231	0.176	0.464
Gemma-2-2b-neogenesis-ita	0.002	0.536	0.079	0.980
Gemma-2-2b-merged	0.951	0.012	0.651	0.075
Gemma-2-2b-it-lora-sql	0.236	0.408	0.000	0.000
Gemma-2-2B-it-4Bit-GPTQ	0.945	0.159	0.000	0.008
Gemma-2-2b-it-distilled	0.482	0.573	0.271	0.276
Llama-2-7b	0.443	0.470	0.148	0.276
Llama-2-7b-chat	0.528	0.868	0.424	0.766
tulu-2-7b	0.125	0.272	0.170	0.888
EYE-Llama_qa	0.657	0.244	0.674	0.968
coma-7B-v0.1	0.953	0.004	0.573	0.731
llama2-Better-Tune	0.136	0.699	0.469	0.860
Llama-2-7B-Chat-GPTQ	0.594	0.603	0.582	0.424
llama-2-7b-logit-watermark-distill-kgw-k1-gamma0.25-delta2	0.349	0.224	0.366	0.976

Table 15: **Sample results.** Pairwise similarity-style scores between target models (rows) and base models (columns). Cells with $1 - p \geq 0.99$ are highlighted in green as same-lineage pairs.